

南京工业大学

2026 届毕业设计（论文）

题 目： 基于神经辐射场和运动感知的图像

去模糊技术研究

专 业： 计算机科学与技术(嵌入式培养)

班 级： 计嵌 2202

姓 名： 倪成龙

指导教师： 刘 斌

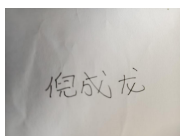
起讫日期： 2025.12~2026.06

2026 年 6 月

南京工业大学本科毕业设计（论文）原创性声明

本人郑重声明：所呈交的毕业设计（论文），是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

论文作者签名：

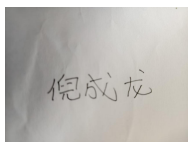


日期： 2026 年 6 月 4 日

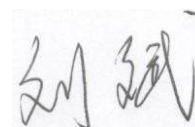
南京工业大学本科毕业设计（论文）使用授权声明

本学位论文作者完全了解学校有关保留、使用毕业设计（论文）的规定，同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权南京工业大学教务处可以将本毕业设计（论文）的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编毕业设计（论文）。

论文作者签名：



导师签名：



日期：2026 年 6 月 4 日

日期：2026 年 6 月 4 日

基于神经辐射场和运动感知的图像去模糊技术研究

摘 要

伴随多视角图像采集与三维内容生产需求的不断上升，相机抖动、离焦以及物体自身运动共同造成的混合模糊已成为视觉重建领域中较具挑战性的问题。传统二维卷积退化模型仅关注单张图像的纹理恢复，难以在多视角输入下保持几何一致性与细节真实感，也难以从模糊观测中重建出稳定、可漫游的三维清晰场景。神经辐射场（NeRF）以连续隐式函数刻画场景的密度与颜色，使从多视角输入中恢复三维场景成为可能，但若不在优化过程中显式建模成像端的退化，仍会被模糊观测误导。本课题在此背景下，以 Deblur-NeRF 项目为基础，对神经辐射场与运动感知相结合的图像去模糊方法进行了系统研究与工程实现。

论文以 LLFF 数据组织、相机位姿读取与归一化为基础，分析并实现了由位置编码、粗细两级 NeRF 网络与体渲染积分组成的清晰场景表示；在此基础上对运动感知稀疏核 DSKnet 进行了细致解析与复现，把每一个模糊像素解释为多条偏移光线渲染结果的加权叠加，使神经辐射场在优化过程中显式建模成像端退化。论文对数据读取、网络构建、损失计算、优化策略、训练监控以及 Streamlit 可视化界面进行了完整的实现，并对关键模块的接口与数据流进行了说明。通过 blurball、blur5grass 等典型场景的功能测试与基于原方法报告的指标分析，验证了运动感知稀疏核相对于基础 NeRF 的有效性，为后续在动态场景与轻量化方向上的扩展工作提供了可复现的工程参考。

关键词：神经辐射场 图像去模糊 运动感知 体渲染 多视角重建

Research on Image Deblurring Based on Neural Radiance Fields and Motion Awareness

ABSTRACT

With the growing demand for multi-view image acquisition and 3D content production, mixed blur caused jointly by camera shake, defocus and object motion has become a challenging problem in visual reconstruction. Traditional 2D convolution-based degradation models focus only on per-image texture restoration and cannot preserve multi-view geometric consistency, nor can they recover a stable, free-viewpoint 3D sharp scene from blurry observations. Neural Radiance Fields (NeRF) represent scenes as continuous implicit functions of density and color, making it possible to recover 3D scenes from multi-view inputs; however, without explicit modeling of imaging-side degradation, the optimization will still be misled by blurry supervision. This work extends per-image 2D degradation recovery to 3D-consistent recovery under multi-view joint optimization. Building upon this background, this thesis presents a systematic study and engineering implementation of an image-deblurring method that combines neural radiance fields with motion-aware sparse kernels, based on the Deblur-NeRF codebase.

Based on the data organization of LLFF, the thesis implements camera pose loading and recentering, followed by a sharp-scene representation built from positional encoding, coarse/fine NeRF MLPs, and volume-rendering integration. A deformable sparse-kernel network (DSKnet) is then analyzed and reproduced, interpreting each blurry pixel as the weighted sum of multiple offset rays rendered through NeRF, so that the radiance field is jointly optimized with an explicit blur-formation model. The thesis provides a complete implementation covering data loading, network construction, loss computation, optimization strategy, training monitoring and an interactive Streamlit interface, and clarifies the interfaces and data flow of the key modules. Through functional tests on representative scenes such as blurball and blur5grass, together with quantitative analysis referenced from the original Deblur-NeRF report, the effectiveness of the motion-aware sparse kernel over the vanilla NeRF baseline is verified, providing a reproducible

engineering reference for future extensions to dynamic scenes and lightweight 3D scene reconstruction.

Key Words: neural radiance fields; image deblurring; motion awareness; volume rendering; multi-view reconstruction

目录

摘 要	III
ABSTRACT	IV
第一章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.3 论文主要内容与结构	4
第二章 相关技术与开发环境	6
2.1 开发环境与数据组织	6
2.1.1 开发环境	6
2.1.2 数据集组织	7
2.2 神经辐射场表示原理	8
2.3 运动感知去模糊建模	8
2.4 关键数学模型与符号说明	9
2.5 本章小结	13
第三章 系统设计与实现	14
3.1 系统总体架构	14
3.1.1 系统需求分析	14
3.1.2 系统流程设计	15
3.2 数据读取与预处理实现	16
3.2.1 位姿归一化与坐标处理	16
3.2.2 随机光线批处理	16
3.3 NeRF 与 DSK 联合网络实现	18
3.3.1 体渲染主网络	18
3.3.2 运动感知稀疏核网络	19
3.4 生成、渲染与评价模块实现	21
3.4.1 损失函数与优化策略	21
3.4.2 渲染输出与文件组织	22
3.5 可视化交互界面实现	25

3.5.1 配置管理模块设计	25
3.5.2 结果对比模块设计	25
3.5.3 交互系统的工程价值	26
3.6 本章小结	29
第四章 实验设计与结果分析	30
4.1 实验环境与数据集设置	30
4.1.1 数据场景与划分方式	30
4.1.2 关键参数设置	30
4.2 评价指标与验证方法	32
4.3 结果分析与系统验证	33
4.3.1 对比实验设计	33
4.3.2 功能测试设计	33
4.3.3 实验风险与误差来源	35
4.4 本章小结	37
第五章 总结和展望	38
5.1 研究总结	38
5.2 工作展望	39
参考文献	40
致谢	44

第一章 绪论

随着数字图像采集设备的普及，三维视觉应用也迅速发展，多视角图像去模糊这个方向已经成为了计算机视觉中一个比较重要的研究问题。本章首先回顾了图像去模糊技术的研究背景和意义，分析了传统方法在多视角场景下存在的不足；然后梳理了神经辐射场及其在去模糊任务上的研究现状；最后给出本文的主要工作和章节安排。

1.1 研究背景与意义

从成像过程来看，模糊不是由于一个原因引起的简单失真。相机在曝光时间内发生抖动就会产生相机运动模糊，场景中物体相对于相机快速移动就会产生局部动态模糊，镜头焦平面与目标深度不一致就会产生离焦模糊。退化在真实数据中常常同时出现，并且会随着空间位置、景深、观察视角的变化而变化，所以很难用一个固定的卷积核来完整地描述。

对于二维图像复原任务，深度学习方法通常将模糊图像直接映射为清晰图像，由网络学习局部纹理的恢复与边缘的增强。该思路在单张图像质量提升任务中已取得较好效果，但当任务从单张图扩展到多视角三维场景时，仅在二维平面做后处理会破坏视角之间的几何一致性。常见的现象是同一物体的边缘在不同视角下出现不同程度的位移与错位，从而难以支撑稳定的三维重建。

随着神经辐射场这一概念被提出，多视角视觉任务在建模方式上也跟着发生了变化。它不再像传统方法那样显式地存点云、网格或者体素，而是用一个连续函数来表示任意一个空间点的密度和颜色，然后依靠体渲染方程把任意视角下的图像生成出来。该表示既是连续的又是紧凑的，可以求微分，因此可以把多视角图像的监督转化为端到端的优化问题来求解。对于图像去模糊任务来说，可微渲染机制也提供了一种新的接口，用来对模糊成像的过程进行建模。

本课题的意义在于把图像去模糊从单纯的二维复原问题提升为三维一致性恢复问题。如果系统可以从模糊多视角图像中恢复出比较清晰的神经辐射场，那么不仅可以得到单个视角的清晰图片，还可以在新的相机位置生成稳定的清晰视图。对于数字文物采集、三维建模、机器人环境感知、虚拟现实素材生成等有良好的发展前景。

从工程角度来讲，Deblur-NeRF 这个项目本身已经把数据加载、模型生成、渲染输出、指标评价、可视化交互这些环节配得比较齐了，属于一个相对完整的研究原型。围绕这个项目做毕业设计，一方面可以对 NeRF 和去模糊算法相关的理论进行梳理，另一方

面也可以锻炼对深度学习工程代码、实验配置管理、日志整理和结果分析等几个方面的综合能力。

近几年来，手机、无人机、巡检机器人、数字文保、沉浸式内容生产等场景的发展速度都比较快，图像和视频采集设备被越来越多地用于低照度、手持拍摄、长曝光、快速运动的环境下。因此模糊退化就逐渐成为影响视觉感知质量、也影响三维重建质量的一个重要因素。一旦输入图像本身比较模糊，不仅会给人带来较差的观感，后面要做的特征匹配、姿态估计、三维建模等步骤的精度也会跟着下降。

传统图像去模糊研究主要是对二维退化过程进行建模，把模糊看作清晰图像和卷积核作用的结果，再通过先验约束或者深度网络恢复出潜在的清晰图像。这类方法在单幅图像复原任务中已经取得了大量成果，但是它们一般不能显式地利用多视角图像之间的几何关系，因此在三维一致性要求较高的场景中仍然存在局限。

神经辐射场技术用连续隐式函数统一表示场景的密度和颜色，从多视角图像中恢复三维场景是可能的。若能在 NeRF 优化过程中同时建模模糊形成机制，那么就可以得到既保持多视角几何一致性、又恢复清晰纹理细节的图像去模糊方案。因此，研究基于神经辐射场和运动感知的图像去模糊技术有明确的理论价值和工程意义。

1.2 国内外研究现状

早期图像去模糊方法大多基于退化模型和图像先验展开，一般认为模糊图像是由潜在清晰图像和模糊核卷积后叠加噪声得到的。研究者用稀疏梯度、全变分、边缘先验、自然图像统计等约束估计清晰图像和模糊核。该类方法具有较强的解释性，但是对于复杂多变的动态环境、含有模糊核的核估计往往会出现纹理干扰、噪音叠加以及遮挡效应导致结果失真等问题。

深度学习方法出现以后，图像去模糊从显式核估计变为端到端映射学习。多尺度卷积网络逐层恢复结构信息，循环神经网络从粗到细的过程中共享参数来降低计算量，生成对抗网络用感知损失和判别器来提高视觉的真实性。这些方法一般不需要对每个位置的模糊核进行估计，因此在真实图像上具有更好的适应性。

然而，二维深度去模糊也存在明显不足。由于此类网络的监督信号要么来自清晰图像，要么来自合成退化图，模型容易记忆训练集的纹理统计，而不是真正理解场景的三维成像过程。当输入扩展为多视角图像时，逐张去模糊在单视角观感上可能尚可，但不同视角之间在颜色、边缘与遮挡关系上的一致性往往难以得到保证，进而影响下游的三维重建结果。

多视角三维重建本质上就是把传统的 Structure-from-Motion、Multi-View Stereo 向神经隐式表示发展。传统的那一类方法对特征点匹配、相机位姿估计以及深度融合的依赖度较高，需要图像中有明显的纹理和稳定的特征点；图像一旦糊起来，特征检测和匹配的稳定性就会下降很多。NeRF 这一类方法因为直接用渲染误差来优化，对显式匹配的依赖小一些，但是输入图像质量本身仍然会传递到最终的场景表示里去。

NeRF 以及围绕它做的后续工作，已经基本说明了隐式体表示在新视角合成中具有较强的能力。Mip-NeRF 这一类方法又把采样锥体和抗混叠的问题考虑进来，使得模型在不同尺度下都能给出更稳定的结果。但是这些工作主要解决的是输入图像清晰时的表示能力和渲染质量问题，对于由于输入模糊造成的监督偏差，并没有专门去处理。

Deblur-NeRF 的核心方法就是把模糊的像素解释成它邻域内几条光线渲染结果的加权和。和把图像去模糊好、再用清晰图训 NeRF 这种两阶段流程相比，联合优化的好处在于模糊核学习和三维场景表示可以互相约束。这里的模糊核不是一个二维滤波器，它和相机位姿、像素位置、视图编号都有关，整体上是一个可形变的采样结构。

从国内外的研究中可以发现，二维图像复原、神经隐式表示、可微渲染正在逐渐融合。未来图像去模糊的研究不单是对一张图片的清晰度进行评价，还会对恢复后的结果是否可以支撑起稳定的三维重建、跨视角一致渲染以及下游的视觉任务进行考虑。因此，基于神经辐射场和运动感知的去模糊技术有较强的研究价值。

二维图像去模糊这一研究路径上，多尺度卷积网络、循环递归网络、生成对抗网络等做法相继被提出。Nah 等人提出的多尺度卷积网络以由粗到细的方式逐级恢复动态场景中的清晰结构^[1]；Tao 等人提出的尺度递归网络在参数规模与跨尺度信息传递之间做了较好的折中^[2]；Kupyn 等人提出的 DeblurGAN-v2 在速度与去模糊质量之间取得平衡，并通过特征金字塔与判别器提升了真实图像上的鲁棒性^[3]。上述工作在单视角图像去模糊上取得了较好的效果，但都未直接处理多视角几何一致性问题。

三维场景表示这一边，NeRF 把空间位置和视角方向作为输入，映射到颜色与体密度，实现了高质量的新视角合成^[4]。此后针对抗锯齿、采样效率、训练加速以及实时渲染等问题，陆续出现了 Mip-NeRF^[5]、Mip-NeRF 360^[6]、Instant-NGP^[7] 与 3D Gaussian Splatting^[8] 等改进做法。这些工作主要解决清晰输入下的表示能力与渲染质量问题，并不针对模糊监督引入的偏差。

为解决多视角模糊输入的问题，Deblur-NeRF 在 NeRF 框架中加入可学习的稀疏模糊核，为每个像素预测若干条偏移采样光线和对应的融合权重，以显式方式建模相机运动

与离焦造成的空间可变模糊^[9]。与先对每张图做二维去模糊再重建 NeRF 的两阶段流程相比，该方法把模糊形成过程直接放入三维场景优化中，使清晰场景重建与模糊核学习互相约束、共同收敛。在此之后，BAD-NeRF^[10]、DP-NeRF^[11]、ExBluRF^[12]、E2NeRF^[13]、PDRF^[14] 等工作沿着该思路在束调整、物理先验、极端运动、事件相机辅助与级联反卷积等方向继续扩展。

DynaMoDe-NeRF 面向动态场景下的局部物体运动模糊，把三维速度、曝光时间与二维模糊核位置建立投影关系，使模糊像素可由运动轨迹上多个清晰辐射场样本来解释^[15]。该工作表明，运动感知去模糊不仅适用于静态多视角场景，也可以扩展到具有时间变化与局部运动的动态场景。沿着这一方向还出现了 DyBluRF^[16]、MoBluRF^[17] 以及面向 3D Gaussian Splatting 的去模糊扩展^[18]，逐步把多视角去模糊问题向动态、实时与轻量化方向推进。综合来看，二维去模糊方法在局部纹理上较为擅长，NeRF 类方法则在多视角一致性上更有优势，Deblur-NeRF 大致是这两条思路融合的产物，沿这一方向开展工程复现与系统整合，对开展三维视觉感知研究具有一定参考价值。

1.3 论文主要内容与结构

本文以目前的 Deblur-NeRF 项目为基础，对基于神经辐射场和运动感知的图像去模糊方法进行了梳理、复现和系统化实现。研究内容主要是对算法原理进行分析，对工程结构有充分的认识，对配置过程有系统化的组织，以及对于界面交互的实现有具体的操作步骤。

本文的主要工作分为以下三个方面：

第一部分对 LLFF 的主要算法流程进行详细描述，包括数据读取、位姿预处理与体渲染采样，并简要介绍所采用的神经网络结构，以厘清项目的核心算法流程；

第二部分围绕 Deformable Sparse Kernel 模块，结合 NeRF 主网络、生成调度与评价指标，构建方法在实现层面的完整说明；

第三部分是将生成、推理、结果分析这三个过程用 Streamlit 界面进行交互化封装，目的是使整个工程的复现效率更高。

论文主要结构如下：

第一章 绪论：介绍课题背景、研究现状与本文的研究内容，阐述运动模糊场景下三维重建所面临的难点，梳理国内外在 NeRF 去模糊方向上的相关研究进展，并在此基础上给出本文的研究思路与主要贡献；

第二章相关技术与开发环境，介绍项目依赖的相关技术、开发环境和数据组织形式，包括 Python 与 PyTorch 开发环境的搭建、LLFF 格式数据集的组织方式、神经辐射场的基本原理，以及运动感知模糊建模的核心思路，为后续章节的系统实现打下理论与工程基础；

第三章重点说明基于 NeRF 与运动感知稀疏核的系统设计与实现，详细阐述了数据预处理流程、运动感知稀疏核网络（DSKnet）的结构与核生成过程、NeRF 主网络的体渲染流程、多损失函数的构成方式，以及基于 Streamlit 搭建的可视化交互系统的功能设计；

第四章讨论实验配置、评价指标、功能验证与结果分析，给出了训练与推理所采用的硬件平台和超参数设置，介绍了 PSNR、SSIM 和 LPIPS 等定量评价指标，并对多个测试场景下的去模糊效果进行了定性与定量的对比讨论；

第五章是对全文工作的总结，提出后续改进的方向，回顾了本文在数据集的建立、模型的训练、系统的开发等各方面所取得的主要成果，并且对先验约束补全、深度感知建模、推理效率优化等层面给出了下一步的研究思路。

第二章 相关技术与开发环境

为了使论文分析和项目实现保持一致，本章从工程环境、数据组织、神经辐射场原理、运动感知模糊建模这四个方面来说明，给后面系统的实现章节提供技术支持。

2.1 开发环境与数据组织

2.1.1 开发环境

本文的实验和系统实现是在当前的 Deblur-NeRF 项目代码基础上完成的，也参考了 DynaMoDe-NeRF 在多视角运动模糊场景下的实验设置。整个系统主要使用 Python 来实现，用 PyTorch 搭建了 NeRF 主网络、运动感知稀疏核网络、体渲染的优化流程。模型参数的更新使用的是 Adam 优化器，训练和渲染会优先放到带 CUDA 的 GPU 环境里跑；CPU 环境也保留下来，用于配置解析、数据加载以及小规模流程上的验证。

项目运行所必须的核心环境配置见表 2-1。

表 2-1 项目开发与运行环境配置

类别	配置内容
开发语言	Python
深度学习框架	PyTorch、torchvision
硬件环境	支持 CUDA 的 NVIDIA GPU，CPU 可用于流程测试
优化方法	Adam 优化器
数据处理	NumPy、imageio、Pillow、OpenCV
评价指标	scikit-image、LPIPS、PyTorch
配置管理	configargparse
可视化与交互	Streamlit、Plotly、Matplotlib
日志记录	TensorBoard、tensorboardX
数据格式	LLFF 风格数据结构，包含图像、相机位姿和景深边界信息

从工程结构上来说，run_nerf.py 是整个系统的主入口，参数解析、模型创建、训练优化、测试渲染、结果保存等基本都从这里开始；NeRF.py 主要定义 NeRF 体渲染网络、DSKnet 稀疏核模块以及相关的前向计算；load_llff.py 负责把 LLFF 格式的数据读进来，包括图像、相机位姿和近远景深边界；metrics.py 用来算 PSNR、SSIM、LPIPS 这一

类图像质量指标；ui.py 这一块是基于 Streamlit 给出一个可视化的交互界面，方便做配置管理、实验启动、结果查看、指标分析等事情。

本文主要研究的是多视角模糊图像条件下神经辐射场重建和去模糊问题。因此环境配置的重点在于保证三类流程可以稳定运行，第一，可以读取多视角图像和对应相机位姿；第二，可以在 PyTorch 中完成 NeRF 和运动感知模块的联合优化；第三，可以对渲染结果进行定量评价和可视化展示。本章所使用的环境配置，能满足本文算法分析、功能测试、系统展示的基本需求。

2.1.2 数据集组织

数据集部分，项目目前主要按照模糊类型来组织。data 目录下分为真实相机运动模糊、真实离焦模糊、真实物体运动模糊、合成模糊数据。其中真实相机运动模糊那块包含了 blurball、blurcoffee、blurstair 等多个场景；真实离焦模糊里有 defocusbush、defocuscake、defocustools 等场景；真实物体运动模糊这一块则提供了 blur5grass 一个场景。分法的好处就是可以比较直接地去验证模型在相机抖动、镜头离焦、局部物体运动等不同的模糊条件下表现。

各个场景大体上是按照 LLFF (Local Light Field Fusion, 局部光场融合) 的数据格式来组织的，一般包含 images 这个文件夹再加一个 poses_bounds.npy 文件。images 里放的是原始输入图像，images_4、images_5 等目录是按不同降采样比例生成出来的图像版本；poses_bounds.npy 是每张图像对应的相机位姿、图像尺寸、焦距和近远景深边界，是 NeRF 做多视角重建和体渲染时比较重要的输入。有些场景下还会存在 sparse 目录，一般与 COLMAP 重建结果有关，可以用来检查相机轨迹，也可以用来重新生成位姿信息。

数据加载这一步，load_llff.py 会把场景图像和位姿文件读进来，然后根据配置里的 factor 参数自动选择或者生成对应分辨率的图像目录。这样做的好处是，渲染质量和计算效率之间可以进行平衡，对于分辨率较高的图像，适当降采样一下可以减轻显存占用和计算开销，对模型调试和实验复现都有好处。

为了评估新的视角生成能力，在本文中我们使用 llffhold 参数来划分视角。llffhold 设为 7 的时候，系统就会把索引为 0、7、14 这样的图像当作测试视图，剩下的图像用来做模型优化。这种划分方式比较简单也比较稳定，在不同的场景下可以保持一致的实验设置，方便后面再去比较渲染结果和评价指标。

根据本地数据规模，blurball 场景有 27 张图像，blur5grass 有 35 张图像，真实离焦场景大致有 30 到 50 张视图。多视角图像一方面给 NeRF 学习场景的几何和外观提供素材，另一方面也给运动感知去模糊模块提供模糊观测和视角变化之间的约束关系。综合来看，目前的数据组织方式比较清楚，对于本文围绕图像去模糊、三维重建、新视角渲染流程做的实验分析是够用的。

2.2 神经辐射场

神经辐射场（Neural Radiance Fields，简称 NeRF）是用隐式神经表示为基础的一种三维场景建模方法，它的主要思想就是用一个多层感知机拟合一个连续的五维函数，把三维空间位置（坐标 x 、 y 、 z ）和观察方向（ dx 、 dy 、 dz 两个角度变量）映射成颜色 c 和体密度 σ 。颜色用来描述该空间点在某一个特定观察方向下的辐射信息，体密度用来表示这个点对光线传播的遮挡程度。由于该表示在三维空间中是连续定义的，理论上可以从任意一个相机位姿出发对场景进行采样和渲染。

和传统的三维重建路线相比，NeRF 不需要把稀疏点云、深度图或者网格模型这些中间产物显式地恢复出来，它走的是另一条路，靠图像重渲染的误差直接去优化神经网络的参数。在相机内外参已知的情况下，每一个像素都可以对应到一条从相机中心射出去的光线。系统先沿光线在近远平面之间采若干个空间点，再用 NeRF 网络预测每个采样点的颜色和密度，最后把这些预测代入体渲染公式做积分，就可以算出该像素的预测颜色。不断地把预测图像和真实图像之间的误差压下去，网络也就慢慢学会了场景的几何结构和外观。

为了使网络对细节纹理和高频变化有更充分的表达，NeRF 一般都会对输入坐标做位置编码。位置编码这一步使用一组不同的频率正弦和余弦函数，把维度较低的坐标映射到一个高维的特征空间，这样多层感知机就更容易去表示比较复杂的空间变化。本项目中三维位置和视角方向都是经过位置编码处理之后再送入模型的，主要目的是使模型在空间结构、边缘细节、视角相关颜色这几块的拟合能力更强一些。

渲染这一部分使用的是粗细两级采样的方式，目的是使计算更有效率。粗网络先对光线做一次初步采样，估算各个采样点的重要性；细网络再根据粗网络给出的权重分布，到贡献比较大的区域里再采样、再渲染一次。这样就将更多的计算资源放在了物体表面以及颜色变化明显的地方，从而提高了渲染质量。目前项目中 NeRFAll 模块基本上就是按照粗细联合采样加体渲染的流程搭起来的。

对于 LLFF 类型的前向场景，项目还支持 NDC（归一化设备坐标）变换，将近远平面的采样范围归一到 $[-1, 1]$ 之间，从而提升前向场景的渲染效率与采样数值稳定性。在采样策略上，固定采样点容易导致采样伪影，因此项目采用扰动采样：训练阶段在样本位置加入随机扰动，使离散采样在统计意义上近似连续积分；测试阶段关闭扰动，以获得更稳定的渲染输出。该设计使 NeRF 既能表达高频细节，又能保持优化过程的稳定。

这里需要特别注意的是，NeRF 的监督信号是直接从输入图像的颜色中得到的，相关原理示意如图 2-1 所示。如果输入图像本身比较清晰，那么渲染误差就可以推着网络去学习真实的场景几何和纹理；但是当输入中存在相机抖动、离焦或者物体运动模糊时，那些糊掉的边缘也会被网络当成真的颜色变化去拟合，重建结果里就容易出现细节丢失、几何不稳定一类的问题。因此，一旦观测是模糊的，只依靠标准 NeRF 渲染模型就不太够用了，还需要再加一个运动感知的去模糊建模机制进去。Deblur-NeRF、DynaMoDe-NeRF 等方法都是围绕这一点展开的，把模糊形成过程和神经辐射场的优化过程结合起来，在模糊输入的情况下新视角合成和场景恢复的质量都能得到一定程度的提高。

2.3 运动感知去模糊建模

多视角图像重建任务中，图像模糊不是图层面的二维退化问题，还会对三维场景表示的学习产生影响。传统二维去模糊一般把模糊图像看作清晰图像经过某种卷积核退化之后的产物，再让神经网络把清晰图像反推回来。但是神经辐射场场景下，每个像素的颜色是由一条穿过三维空间的相机光线积分得到的，如果输入视图中有相机抖动、有离焦、有局部物体运动，那么一条理想光线就很难解释观测到的这种模糊像素了。因此，在体渲染过程中，需要把曝光期间可能出现的各种成像状态都考虑进去。

Deblur-NeRF 的主要思想就是把模糊的像素看作是由几条相关采样光线渲染结果的加权组合，而不是直接在二维图像平面上做卷积。对于一个等待被优化的像素，模型就不再只沿着中心光线采样了，而是再预测出若干条带有空间偏移的辅助光线，分别经过 NeRF 网络得到对应的颜色。之后系统再根据网络学到的融合权重，把这几个颜色做一次加权求和，得到和输入模糊图像对应的渲染结果。因此模糊建模就从原来的图像卷积变成了光线偏移、权重分配和三维体渲染三者联合优化的问题。

该建模方式与普通的二维去模糊网络相比，差别还是很大的。二维去模糊网络一般在图像域上做局部纹理映射，恢复结果主要依靠图像上下文来约束；而 NeRF 这边做稀疏核融合的时候，每一条偏移光线都要走过同一个隐式三维场景再渲染出来。换句话说，模糊核学到的东西不只是一定要能拟合当前图像里的像素，它还要和多视角的几何关系对得

上。当采样的光线偏移量过大或者方向不当时，那么渲染出来的颜色就容易出现误差，进而影响到网络的训练结果，使得网络只能对偏移的位置进行调整和融合权重的优化。因此，这种方法可以在去模糊的过程中带进三维一致性的约束，不至于完全依靠二维图像先验。

从成像机理的角度来说，运动感知的去模糊建模可以将好几种模糊现象统一解释。相机运动模糊本质上就是曝光期间相机姿态发生了一点点的小变化，使得同一个像素接收到来自邻近视角方向的光线贡献；离焦模糊这边，像素颜色可能是由焦平面附近若干空间位置一起决定的；至于动态物体造成的模糊，则是局部区域在曝光期间产生了位移，让观测颜色变成了不同时间状态下的混合。模糊来源虽然不同，但是都可以近似地看成多条候选光线颜色按权重融合在一起。因此把这一切统一在神经渲染框架里，去预测稀疏的采样光线，对于各种类型的模糊场景都能较好地适应。

运动感知部分主要由 **DSKnet** 模块完成。其接收的输入包括三类：图像索引、像素坐标，以及规范核的采样点（相机位姿用于在另一步生成光线，不属于网络输入）。在此基础上，**DSKnet** 通过一组可学习的图像嵌入与空间位置编码，推断每个像素所对应的稀疏采样核。图像嵌入主要刻画不同输入视图之间的模糊差异（例如不同照片存在方向与程度不同的相机抖动）；空间位置编码则刻画同一图像内部不同区域退化方式的变化，使模型在面对空间非均匀模糊时仍有足够的表达力。

为了控制计算开销，项目中并没有给每一个像素都建立一个密集的二维卷积核，而是用少量代表性的采样点来构成一个稀疏核。`kernel_ptnum` 这个参数是用来控制每个像素对应多少条采样光线的，比较常用的设置是 5。如果一个像素对应的稀疏核里包含 K 个采样点，那么这个像素的模糊颜色就可以表达成式(2-1)所示。

$$\hat{C}(r) = \sum_{i=1}^K w_i C(r_i) \quad (2-1)$$

其中， r_i 表示第 i 条偏移采样光线， $C(r_i)$ 是这条光线经过 **NeRF** 体渲染得到的颜色， w_i 为对应的融合权重，通过归一化操作使所有权重之和为 1。从公式中可以看出，最后用来监督的信号并不是直接约束某一条中心光线，而是去约束多条候选光线融合之后的整体颜色。

DSKnet 的输出结果是由两部分组成的，分别为采样点偏移量和融合权重。在最基础的设定下，`delta_pos` 表示的是采样点在图像平面上的二维偏移，`weight` 表示的是每条采样光线对最终颜色贡献的大小。当启用空间可变平移建模的时候，网络还会再输出一个

`delta_trans`，来模拟曝光期间相机位置上的细微变化。之后模型会结合像素坐标、相机内参再加上预测出来的偏移，重新算一遍光线方向，再利用对应视图的相机位姿把光线变换到世界坐标系里。通过这一套流程，二维上的像素偏移就被转成了三维空间中的渲染光线，模糊核学习和相机几何之间也建立了联系。

由于稀疏核自由度很高，如果不加以约束，网络就会用很大的光线偏移来降低训练误差，从而破坏场景的几何一致性。为了缓解上述问题，在 DSKnet 中加入了 `align` 正则项，对中心采样点的偏移和视点平移的幅度做了些限制。这一项可以使得采样光线尽量围绕原始像素附近分布，避免模型给出一些不太合理的偏移解释。在配置文件中配置了三个参数，`kernel_align_weight` 表示正则化系数的权重，`align_start_iter` 为正则化生效时间点，`align_end_iter` 同样也起到了相似的效果。

项目中还存在着一个色调映射模块，用来解决线性渲染的颜色和真实图像颜色空间不一致的问题。NeRF 网络输出的颜色一般是在模型内部连续表示空间中的，而真实图像在采集过程中会受到相机响应、曝光、伽马校正等影响。`tone_mapping_type` 这个配置参数是用来指定颜色映射方式的，示例配置中一般是用 `gamma` 映射，使渲染结果更接近真实的低动态范围图像，监督信号上的一致性也就更好一些。

因此运动感知去模糊建模的本质就是把模糊像素解释为曝光过程中多光线观测的结果，在神经辐射场优化过程中同时学习场景表示、采样偏移和融合权重。如图 2-2 所示。相比于传统的二维图像复原方法，本文所用的方法可以利用多视角几何约束来建模相机运动、离焦、局部物体运动等模糊场景。DSKnet 模块就是这个思想的实现，用稀疏采样的核来减小计算量，用图像嵌入、空间编码、正则约束来提高模糊建模的灵活性和稳定性。

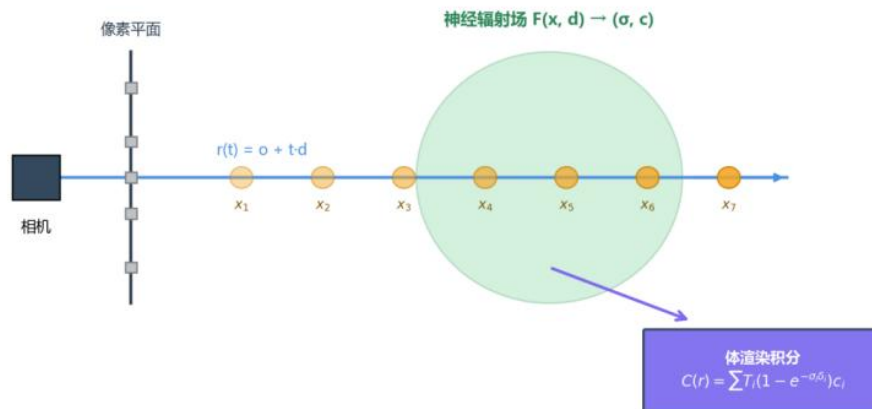


图 2-1 神经辐射场体渲染原理示意

2.4 关键数学模型与符号说明

为了便于后面说明，本文对神经辐射场渲染和运动感知去模糊过程中用到的数学模型进行概括。本文把模糊图像看作曝光时间内多个清晰成像状态的融合结果，用 NeRF 主网络和运动感知稀疏核模块一起完成建模。

相机光线可以如式 (2-2) 所示。

$$r_{i,p}(s) = o_i + s d_{i,p}, \quad s \in [s_n, s_f] \quad (2-2)$$

上面这个式子中， o_i 代表相机的光心位置， $d_{i,p}$ 代表该像素对应的光线方向， s 用来表示采样的深度。

NeRF 用多层感知机来表示连续三维场景，输入为空间位置和观察方向，输出为颜色和体密度，如式 (2-3) 所示。

$$F_\theta(x, d) = (c, \sigma) \quad (2-3)$$

式 (2-3) 中 x 表示的是采样到的三维点， c 是 RGB 颜色， σ 是体密度。如果面对的是动态的场景，在前面加上时间变量 t ，就可以写成 $F_\theta(x, d, t)$ 的形式。

将体渲染离散化之后，假设一条光线上一共采到了 N 个点，那么这条光线最终渲染出来的颜色如式 (2-4) 所示。

$$\hat{C}(r) = \sum_{k=1}^N T_k \alpha_k c_k \quad (2-4)$$

其中 α_k 是第 k 个采样点的不透明度， T_k 是该点之前所有采样点的累积透射率。这个公式和项目代码里 `raw2outputs` 函数所做颜色和权重计算是一致的。

本文采用的是模糊图像的稀疏光线融合建模。假定每个像素对应若干条偏移采样光线，光线在图像上的二维偏移记作 Δp_{ij} ，对应的融合权重 w_{ij} ，那么预测出来的模糊颜色就如式 (2-5) 所示。

$$\hat{B}_i(p) = \sum_{j=1}^M w_{ij} \hat{C}(r_{i,p+\Delta p_{ij}}), \quad \sum_{j=1}^M w_{ij} = 1 \quad (2-5)$$

该公式就是运动感知去模糊建模的主要公式。它说明模型并不是直接在二维图像上进行卷积，而是预测多条偏移光线，然后对这些光线经过 NeRF 渲染后的颜色进行加权融合。

模型优化目标可以简化表示为式 (2-6) 所示。

$$\mathcal{L} = \sum_i \sum_{p \in \Omega_i} \|\hat{B}_i(p) - B_i(p)\|_2^2 + \lambda_{\text{align}} \mathcal{L}_{\text{align}} \quad (2-6)$$

上式中 $B_i(p)$ 为输入模糊图像的真实颜色， $\hat{B}_i(p)$ 为模型预测的颜色， $\mathcal{L}_{\text{align}}$ 这一项用来限制采样光线偏移不能太大，从而提高优化阶段的稳定性。

综上所述，本文方法的数学流程可概括为：先由相机参数生成像素光线，再用 NeRF 进行体渲染得到清晰颜色；然后通过运动感知稀疏核预测偏移光线与融合权重，最后重建出与输入模糊图像一致的观测结果，整体流程如图 2-2 所示。

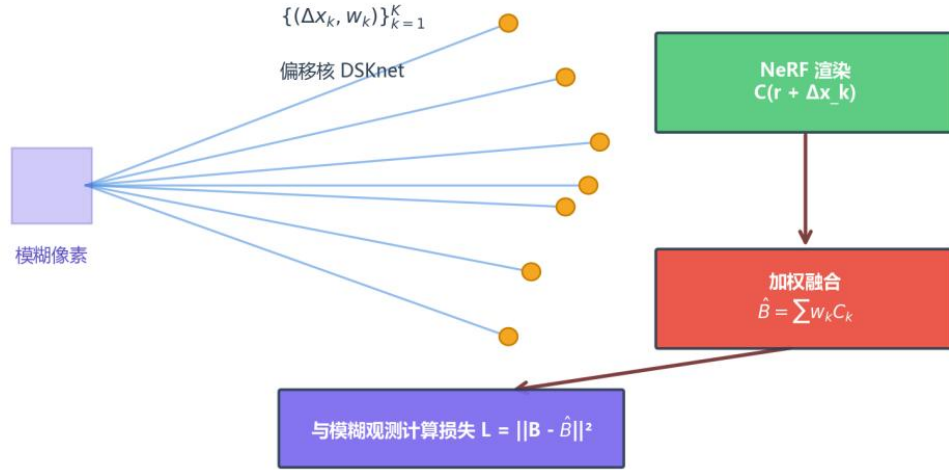


图 2-2 运动感知稀疏核 DSKnet 模糊建模示意

2.5 本章小结

本章主要从开发环境、数据组织、NeRF 表示原理、运动感知模糊建模、关键数学模型这五部分来介绍项目的工程技术基础。从图中可以看出，目前工程并不是简单地调用一个网络，而是围绕数据、采样、优化、评价、界面构建了一个比较完整的实验链路。

第三章 系统设计与实现

在确定相关技术基础之后，本文从工程实现的角度说明项目是如何组织生成、渲染、评价、交互的，并分析各个核心模块在代码中所承担的职责。

3.1 系统总体架构

3.1.1 系统需求分析

可复现性是系统最开始就需要考虑的一个需求。每次运行之后都应该有一个可以追踪的记录，即配置、命令、运行日志、各项指标和模型权重。如果只保留最终图片，而把参数丢掉，那么后续就很难判断这张图对应哪一组超参；反过来，只保留权重而没有配置，重新做渲染或者继续往下优化也会比较麻烦。

系统还需要满足结果可解释的需求：去模糊模型的输出不是简单的分类标签，而是图像、视频、深度图与定量指标的组合（如图 3-1 所示）。因此，可视化界面需要将这这些输出统一组织、对齐展示，从而便于用户直观比较去模糊效果与基础 NeRF 渲染结果之间的差异。

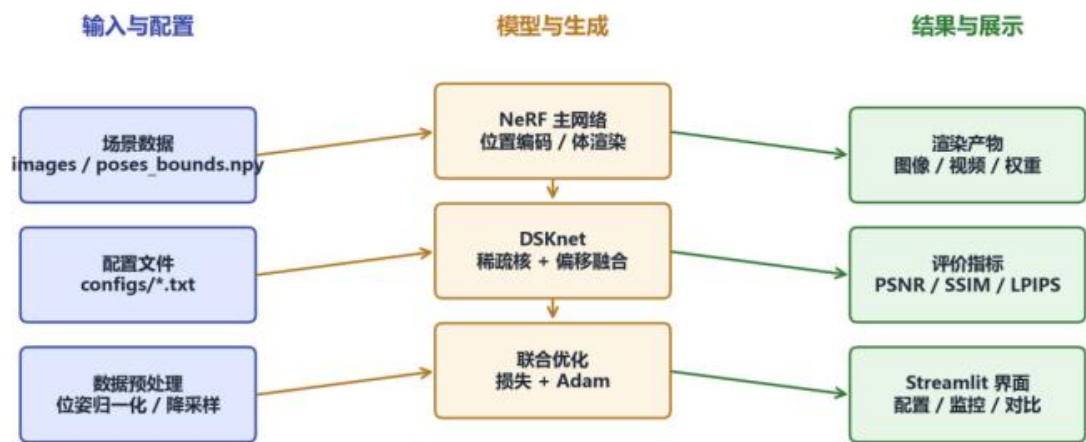


图 3-1 系统总体架构图

系统需要实现的功能是，在基础的 NeRF 和 Deblur-NeRF 两种模式之间自由切换。前一种主要用来搭建基线，方便快速调试；后一种则用来把运动感知稀疏核加进去。两种模式的数据加载、光线采样、体渲染、指标计算等底层逻辑尽量使用同一个代码，否则后期改动会很不方便。

从可用性需求来说，毕业设计所采用的算法不仅要能够运行，而且要能够使结果易于展示和解释。Streamlit 界面要能实现创建配置、启动实验、加载模型、图像对比、指标分析等操作，使答辩或者演示时可以清楚地看到项目运行的过程。

3.1.2 系统流程设计

走到配置阶段的时候，使用者要先想清楚一件事，本次实验是完整的生成、快速调试，还是只是想拿已有的模型再做一次渲染。属于完整生成的，`N_iters` 要设得高一些，DSKnet 相关参数也要尽可能完整；如果只是快速调试，则可以把 `N_rand`、`N_samples`、`N_importance`（即每次迭代随机采样的光线数、粗网络每条光线的采样点数与细网络追加的精细采样点数）都往下调；只渲染已有模型的情况，则需要先确认对应实验目录里既有检查点、也保留了原始配置。

进入生成阶段之后，系统内部有两个时间尺度需要特别注意，一个是迭代步长 `i`，另一个就是 `global_step`。`i` 主要用来控制当前在循环里的位置，决定输出频率；`global_step` 则用在学习率衰减和 TensorBoard 日志记录这两件事上。如果续训，那么 `global_step` 就从之前的检查点恢复过来，保持学习率以及日志的连续。

本阶段系统把模型切换到 `eval` 模式，把梯度计算关掉。一方面可以减少显存占用，另一方面也可以避免 BatchNorm 或者 Dropout 这些优化行为对结果的影响。虽然目前 NeRF MLP 中并没有复杂的归一化层，但是将 `generate` 和 `eval` 两个分支分开，仍然是比较好的工程习惯。

异常恢复工作也应该纳入到系统的流程当中。由于显存不足、缺少依赖、中途断电等原因造成 NeRF 训练中断。项目用定期保存 tar 检查点、自动加载最新的权重的方法实现可以从已经完成的进度点处重新起跑的目的。根据界面层的实验目录状态，使用户知道一次实验是否已经开始、正在产生结果，还是结果已经出炉了。

系统的整体流程可分为六个步骤。第一步准备数据，确保场景目录下 `images` 目录与 `poses_bounds.npy` 文件均已存在；第二步选择或创建配置文件，设置 `exname`、`datadir`、`factor`、采样数量与模糊核参数；第三步构建神经辐射场主网络与数据驱动的稀疏核网络；第四步循环地随机采样光线、渲染、计算损失并更新参数；第五步定期评估，对测试集渲染并计算图像质量指标；第六步呈现结果，将训练日志、渲染图像与指标输出到 Streamlit 界面，便于查阅与对比。

整个系统的真正核心部分是由 `run_nerf.py` 来完成的。脚本先解析参数使用 `config_parser`，然后用 `load_llff_data` 加载数据，接着根据 `kernel_type` 要不要构建 DSKnet，最后创建 `NeRFAll` 实例化，进行生成或者渲染。由于所有的实验行为都是由参数控制的，因此这个脚本既可以跑完整个训练，也可以用来快速验证或者单独渲染。其余几个模块按功能划分，`NeRF.py` 实现 NeRF 主网络、色调映射和 DSKnet；`load_llff.py`

主要做 LLFF 场景图像、相机位姿读取和渲染路径生成；`ui.py` 更偏向用户交互一侧，会扫描实验目录、启动训练、调用推理、做结果对比。

3.2 数据读取与预处理实现

3.2.1 位姿归一化与坐标处理

相机位姿处理在 NeRF 系统中属于比较重要的一个环节。由于 NeRF 是直接三维坐标作为网络的输入，如果场景的坐标范围很大，或者相机分布明显偏离原点，那么网络的拟合难度就会增大，训练稳定性也会受到影响。本项目使用 `load_llff.py` 打开 `poses_bounds.npy` 文件来完成相机位置姿态、图像大小、焦距以及场景界线统一设置工作，以便于后面产生光线并进行体渲染。

在 data 加载阶段，首先读取 `poses_bounds.npy` 文件中的数据，然后根据文件中提供的相机外参、图像高度、宽度和焦距等信息将其整理出来。再下来，系统会根据配置文件中的 `factor` 参数选择或者生成对应分辨率的图像目录，并同步修正图像尺寸和焦距，保证相机内参和实际输入图像的分辨率一致。如果光缩放图像、内参不一起调整，生成的光线方向就会跑偏，进而影响到场景几何重建。

为了使训练更加稳定，项目对相机位姿做了重心归一化的处理。`poses_avg` 用来计算平均的相机姿态，`recenter_poses` 把所有的相机位姿都变换到以这个平均姿态为中心的坐标系下。这样可以将场景坐标范围缩小一些，使相机分布更集中，对 MLP 在有限的位置编码频率下学到空间变化是有帮助的。除此之外，系统还会对 LLFF 数据里的坐标轴重新排一下，以便适应项目内部用的那一套相机坐标约定。

场景的近远边界会使得采样效率以及渲染质量受到影响。`poses_bounds.npy` 文件中记录下来的近远边界，是用来确定光线采样范围的。项目这边通过 `bd_factor` 这个参数对边界做一次缩放，目的是使相机平移和景深范围都落在数值上比较稳定的一个区间。当系统默认使用 NDC 模式时，一般会把 `near` 和 `far` 固定下来；如果改成 `no_ndc`，则会按照真实的边界重新设置采样范围。设置边界合理可以减少无效空间被采样到，训练效率也能提高一些。

等位姿和边界都处理完毕之后，`load_llff_data` 函数就会按照 `llffhold` 参数把训练与测试视图划分开来，如图 3-2 所示。`llffhold` 取 7 的时候，系统每隔一段时间就抽取一部分图像作为测试视图，剩下的图像作为训练用。该切法简单、稳定，适合于不同的实验条件下保持一致的实验设置。



图 3-2 据划分与配置界面

3.2.2 随机光线批处理

跟普通图像网络的做法不同的是，NeRF 在训练时一般不会把整张图像一次性地塞进网络里，而是从全部训练图像中随机抽一批光线来进行优化。当前项目在 `run_nerf.py` 中会先创建训练数据字典，把所有的训练图像像素光线展平成一个一维集合，每次迭代都会从里面随机选择 `N_rand` 条光线参与前向渲染和损失计算。一方面可以降低单次显存占用，另一方面也可以使随机优化的效果更好。

训练数据字典里有 `rays`、`rgbsf`、`images_idx`、`rays_x` 和 `rays_y` 这些字段。其中 `rays` 是光线的原点和方向，`rgbsf` 是对应像素的目标颜色，`images_idx` 是光线来自哪张图像。基础 NeRF 主要依靠光线和颜色监督，而 DSKnet 还要用图像索引来选择对应的图像嵌入和相机位姿，因此该数据组织可以同时支持普通 NeRF 模式和运动感知去模糊模式。

`rays_x` 和 `rays_y` 保存像素在图像平面中的坐标位置，对 DSKnet 比较重要。由于模糊核偏移是在图像平面上定义的，网络需要知道当前光线对应的像素位置，才能预测空间可变的采样偏移和融合权重。项目在创建像素坐标的时候加上半像素偏移，使光线更准确地通过像素中心。

训练时系统会用同一个随机索引来对所有的光线数据进行打乱，使光线、像素坐标、目标颜色、图像索引始终对应。当一个周期内所有的光线都被取完之后，数据就会被重新随机排列一次，模型在长期优化中就会看到来自不同视角、不同空间位置的样本。对于模糊场景来说，随机采样机制对 DSKnet 来说是有利的，它可以帮助 DSKnet 学习图像、区域上模糊核的变化。

该项目中存在一个渲染路径生成函数，训练好之后可以生成测试视图、路径视频等。除了标准的螺旋路径之外，在代码中还保留了 EPI (Epipolar Plane Image, 极平面图像) 这样的路径设置，可以方便地从不同的角度来检查场景重建的效果和去模糊的效果。这说明项目本身所关注的不仅仅是训练流程，对后续结果展示和实验分析也做了相应的考虑。

3.3 NeRF 与 DSK 联合网络实现

3.3.1 体渲染主网络

目前项目体渲染主网络由 Embedder、NeRF、NeRFA11 三个部分组成。其中 Embedder 对三维位置和视角方向进行位置编码，提高网络对高频细节的表达能力；NeRF 类为标准多层感知机结构，预测采样点的颜色和体密度；NeRFA11 在更高层封装粗细两级网络、光线渲染、层次采样、色调映射等流程。

NeRF 主网络采用多层感知机结构。按 NeRF 原论文的默认配置^[4]，粗网络与细网络均为 8 层、每层 256 维的 MLP，输入为空间位置与观察方向，输出为 RGB 颜色与体密度。当 `use_viewdirs` 被启用时，网络对密度与颜色的预测分到两条不同分支：位置分支先估算体密度，随后与方向编码拼接，再由颜色分支预测视角相关的 RGB，从而显式建模视角相关的光照与高光效应。

渲染这一步，系统会先沿着每条光线采若干个空间点，再把这些点送进 NeRF 网络得到颜色和密度的预测。接下来，`raw2outputs` 函数会根据采样间距和密度算出透明度、累积透射率和采样权重，最后就得到了这条光线对应的颜色、深度和累积透明度。这个过程就是 NeRF 体渲染的核心，也是后面图像重建损失的直接来源。

为了提高渲染效率，项目用粗细两级采样的方式。粗网络先在近远平面之间做一次初步采样，算出每个采样点的贡献权重；当 `N_importance` 设得大于 0 时，`sample_pdf` 会按粗网络给出的权重，在贡献较高的区域里补一批采样点，再把这些点交给细网络做二次渲染。物体表面附近权重较大，因此细网络可以将更多的计算资源分配到几何边界和纹理变化明显的地方，渲染质量自然也会随之提高。

为了控制显存的使用，项目在前向渲染这一步使用了分块计算的方式。`mlpforward` 会把采样点展平，再按 `netchunk` 这个大小分批送进 MLP；`render` 函数则按 `chunk` 把光线再分块去渲染。因此，在不改变渲染结果的前提下，可以将单次前向传播的显存峰值降低，对于采样点数量较多的 NeRF 模型来说是比较合适的。

项目中除了 raw_noise_std、white_bkgd、ToneMapping 这一类通用的 NeRF 参数之外，还有其他一些通用参数的设定，如 perturb、lindisp 等等。raw_noise_std 在训练阶段可以给密度预测加入噪声，起到一定的正则化作用；测试阶段一般设为 0，保证输出稳定。ToneMapping 是处理线性颜色和真实图像颜色空间之间差异的，使渲染结果更接近输入图像的监督信号。

总体来说，体渲染主网络这一块的职责就是学习清晰场景的隐式三维表示。训练阶段随机挑选出图像中的光线来加以改善，测试阶段系统针对视图或者渲染路径里的测试图像以及路径视频每帧进行渲染。该模块给后面运动感知稀疏核网络提供颜色渲染的基础。

3.3.2 运动感知稀疏核网络

运动感知稀疏核网络用 DSKnet 来实现。它的作用就是，在 NeRF 已经学到的清晰辐射场的基础上，再把模糊形成的过程也建模出来。DSKnet 不是简单的在二维图像上做卷积，而是给每个像素预测出若干条偏移采样光线，并赋予各自的融合权重；之后再让这些光线分别经过 NeRF 渲染，所得到的颜色按权重融合，最终对应到输入模糊图像的预测颜色，整体数据流程如图 3-3 所示。

DSKnet 接收的输入包括三个：图像索引、像素坐标，以及规范核采样点（相机位姿用于光线生成，不直接进入网络）。其中图像索引用于选择对应的图像嵌入向量，像素坐标用于构造空间位置特征，规范核采样点则为稀疏核中各个偏移位置提供初始分布。经过图像嵌入与空间位置编码之后，DSKnet 既能描述不同视角之间的模糊差异，又能刻画同一图像内部不同区域的空间可变模糊。

网络的输出主要包括采样偏移与融合权重两部分。当启用空间可变平移建模时，网络会额外预测一项，即光线原点上的微小平移。融合权重在输出前先经过 softmax 归一化处理，以保证同一像素内各采样点的权重之和为 1。记第 j 条偏移光线为 r_j ，对应的权重为 w_j ，则模糊颜色可由式 (3-1) 给出。

$$C_{\text{blur}} = \sum_{j=1}^M w_j C_{\text{sharp}}(r_j), \quad \sum_{j=1}^M w_j = 1 \quad (3-1)$$

M 为稀疏核采样点的数量，一般用 kernel_ptnum 来控制。该公式说明模型内部学到的仍然是清晰的辐射场，只是监督目标对应的是多条偏移光线融合后的模糊观测颜色。

如果再借鉴一下 DynaMoDe-NeRF 那种运动感知的想法，就可以把真实运动模糊理解成曝光时间内物体的位置变化在图像平面上的投影。设物体在时刻 t 的三维位置为 o_r^t ，速度为 $v(t)$ ，曝光时间为 τ ，则曝光轨迹上的采样点可以简化为式 (3-2) 的形式。

$$o_j^t = o_r^t + \frac{j-(m+1)}{2m} \tau v(t) \quad (3-2)$$

式中 m 控制的是曝光轨迹两侧的离散采样数量。该公式说明模糊核与曝光时间和物体运动有关。当前项目并不直接估计连续的三维速度，而是用 DSKnet 直接学习图像平面上的光线偏移和权重，因此可以把它看作对上面那个物理成像过程的一个轻量近似。

为避免训练初期清晰场景与模糊核同时随机初始化所导致的优化不稳定，项目采用先基线、后稀疏核的训练策略。在 `global_step` 小于 `kernel_start_iter` 时，系统将 `force_naive` 置为 `True`，暂时绕开 DSKnet，仅对基础 NeRF 进行单独训练；待场景几何具备一定初步表示之后，再进入运动感知稀疏核与 NeRF 主网络的联合优化阶段，从而显著提升训练的收敛性与稳定性。

DSKnet 开始工作的时候，每一个训练光路都被看作是 `kernel_ptnum` 条偏移光线。渲染出所有偏移光线，并使用网络预测得到的权重来加权融合模糊的颜色。粗网络输出的 `rgb0` 也用同样的方式融合并参与损失计算，从而保证粗细两级网络都受到模糊监督的约束。

因此，NeRF 主网络用于表示清晰的三维场景，DSKnet 用于刻画模糊的成像过程，二者共同构成从清晰辐射场到模糊观测图像的可微渲染链路，使系统能够在模糊输入条件下同时优化场景表示与去模糊模型，如图 3-4 所示。

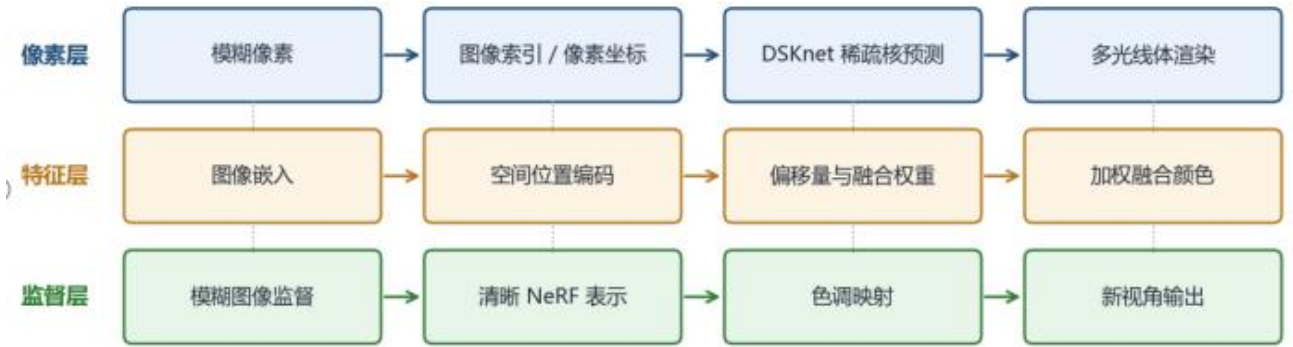


图 3-3 Deblur-NeRF 模糊建模数据流程

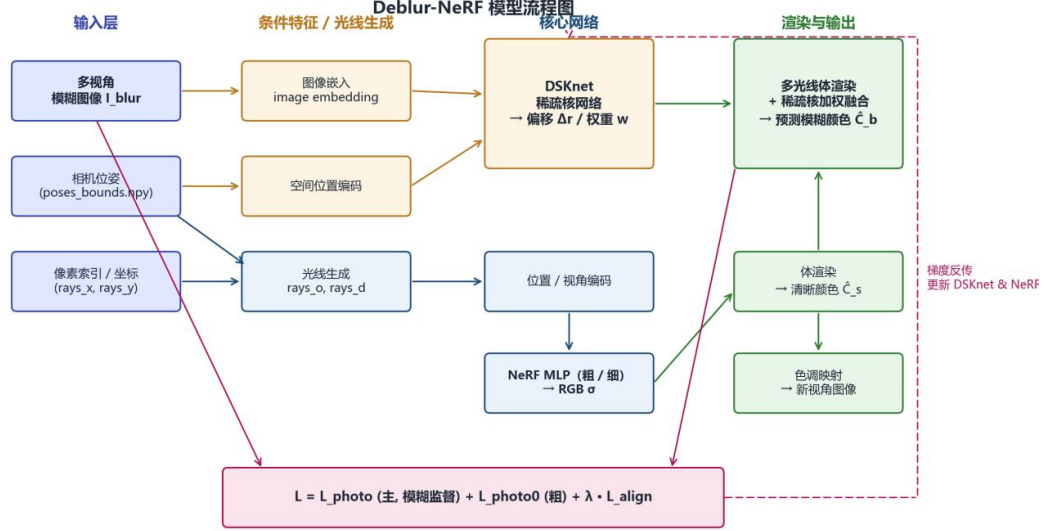


图 3-4 Deblur-NeRF 模型流程图

3.4 生成、渲染与评价模块实现

3.4.1 损失函数与优化策略

生成的时候 `target_rgb` 是 `iter_data['rgbsf']`，也就是输入的模糊图像的像素颜色。对于 Deblur-NeRF 来说，这一点非常重要，模型并没有直接看到清晰的监督图像，而是通过内部模糊合成的过程，使清晰的辐射场间接适应模糊的观测。结果是由于模型结构假设和多视角约束而来的，不是由逐像素的清晰标签所得到的。

当 `force_naive` 为真时，NeRFA11 这部分就直接跳过 DSKnet，把原来的光线渲染到后面。该阶段除了保持场景几何以外，也给之后使用 `depth_embed` 提供了可以利用的深度估计基础。虽然目前的配置中并没有真正启用深度嵌入，但是代码结构上其实已经给这种两阶段信息传递留好了空间。

每次迭代时依次执行 `optimizer.zero_grad`、`loss.backward` 与 `optimizer.step`。由于 DSKnet 与 NeRFA11 被封装在同一优化器中，因此 NeRF 主网络、细网络、图像嵌入、规范核位置与 DSKnet MLP 参数会被一起更新。联合优化是该方法相对于两阶段后处理方法的关键差异：模糊核学习与三维场景表示可以在同一目标下相互约束、共同收敛。

生成过程中主损失 `img_loss` 用来衡量细网络（或者最终融合输出）和目标模糊颜色之间的误差，对应式 (3-3)；`img_loss0` 用来衡量粗网络输出和目标颜色之间的误差，对应式 (3-4)。这两项同时存在的好处是，可以让粗网络也受到有效的监督，从而给层次采样提供一个比较合理的权重分布。如果只监督细网络，粗网络的采样分布就会退化，进而影响后面的重要性采样质量。

$$L_{\text{photo}} = \frac{1}{B} \sum_{r=1}^B \|C_{\text{blur}}(y_r^t) - C_{\text{obs}}(y_r^t)\|_2^2 \quad (3-3)$$

上面的式(3-3)对应的是细网络主损失 L_{photo} ；下面的式(3-4)则是粗网络对应的 L_0 。

$$L_0 = \frac{1}{B} \sum_{r=1}^B \|C_0(y_r^t) - C_{\text{obs}}(y_r^t)\|_2^2 \quad (3-4)$$

align 正则项是由 DSKnet 返回的 align_loss。它由中心采样点偏移的绝对值均值和平移量绝对值均值组成，平移量项乘以 10 来加强约束。该项的意义在于使规范核中的参考点尽量对应原始像素，避免所有采样点整体漂移。其形式如式(3-5)所示。

$$L_{\text{align}} = \text{mean} |\Delta y_0| + 10 \cdot \text{mean} |\Delta t_0| \quad (3-5)$$

尽管项目配置中给出了 kernel_prior_weight、kernel_sparsity_weight 等参数，但是目前的生成循环不会把这些参数硬塞进损失里，它走的是另一条思路，根据 extra_loss 返回的键去动态匹配对应的权重。由于 DSKnet 目前实际只返回 align，所以真正纳入训练的也就只有 align 这一项。当然，该接口给未来留出了扩展余地，完全可以将 DSKnet 也返回 sparsity 或者 prior，再交由生成循环自动加进损失里。项目实际所用的总损失形式可以参考式(3-6)。

$$L_{\text{project}} = L_{\text{photo}} + L_0 + \lambda_{\text{align}} \cdot L_{\text{align}} \quad (3-6)$$

如果采用 DynaMoDe-NeRF 的动态场景设定，那么损失函数中可以加入核平滑、三维速度幅值、三维方向一致性、二维速度投影、时间平滑等正则项，如式(3-7)所示。模糊核平滑项的形式见式(3-8)。本文目前没有实现这些附加项，但是该公式给后面扩展动态模糊约束提供了一个清晰的接口。

$$L = L_{\text{photo}} + \lambda_1 \cdot L_{\text{kernel}} + \lambda_2 \cdot L_{3\text{dm}} + \lambda_3 \cdot L_{3\text{dd}} + \lambda_4 \cdot L_{2\text{dv}} + \lambda_5 \cdot L_{\text{tv}} \quad (3-7)$$

$$L_{\text{kernel}} = \|K_{\text{pred}} - K_{\text{gt}}\|_2, \quad \nabla w = \|w_{j+1} - w_j\| \quad (3-8)$$

学习率采用指数衰减形式更新优化器参数组中的 lr。对于 NeRF 这一类长期训练任务，前期较大的学习率有利于快速建立粗略几何，后期较小的学习率有利于细化纹理与抑制震荡。该策略实现简单、效果稳定，是 NeRF 系列复现代码中的常用做法。具体收敛曲线见第四章 4.3 节。

3.4.2 渲染输出与文件组织

渲染输出可以作为最后的展示，也可以在生成过程中起到一定的诊断作用。举几个例子，测试图像整体发黑，很可能是密度预测或者 near/far 设置有问题；图像颜色偏灰，和 tone_mapping_type、白背景等设置有关；几何结构整体变形，需要回过头去检查相机位姿、NDC 设置或者数据缩放是否出了问题。

深度图和视差图也具有诊断价值。虽然论文主题是图像去模糊，但是 NeRF 的几何质量会影响新视角渲染。如果 RGB 图像看起来很清晰，但是深度图混乱，说明模型主要记

住了生成视图的纹理，而没有学到稳定的三维结构。因此路径视频和深度视频应该和 RGB 结果一起观察。

渲染 `multi-points` 单独输出的子采样点结果可以用来观察模糊核到底做了什么。如果不同采样点最后渲染出来的结果差别很小，那么基本可以认为核没有学到有效的偏移；如果某个采样点的权重一直占据绝对优势，那么模型就退化成了单光线渲染；如果几个采样点呈现出有规律的位移、权重分布，那么才更符合预期里运动模糊建模应该的样子。

生成期间的测试集输出保存在 `logs/expname/testset_iter` 目录下，每张渲染图像按三位编号保存为 PNG。如果执行 `render_only` 并且 `render_test` 为真，输出目录命名为 `renderonly_test_step`；如果渲染路径视频，则生成 `video.mp4` 和 `video_disp.mp4`。清晰的目录命名有利于界面层自动扫描最新的结果。

`test_metrics.txt` 用来保存配置信息以及每次测试的指标记录。指标行有 `iter`、`globalstep`、`MSE`、`PSNR`、`SSIM`、`LPIPS`。在 `ui.py` 中的 `parse_test_metrics` 和 `parse_test_metric_records` 函数中解析这些文本记录来生成曲线以及质量指标展示。

TensorBoard 的指标以及 log 文件一起用作一个整体。前者更适合在生成过程中实时观察 `loss` 和 `PSNR` 曲线的变化；后者比如 `test_metrics.txt` 更适合做界面层解析，也更适合在论文中直接整理使用。两种记录方式同时存在，可以将整个实验结果的可追溯性再加强一些。

`config_parser` 这一部分中参数很多，把数据路径、网络结构、生成迭代、采样数量、模糊核设置、渲染选项、日志频率都包含在内。相比只提供固定参数的脚本来说，本文设计的脚本更加方便复现研究方法，在实验场景发生变化、参数被消融的时候，用户可以自由改变脚本中的各种参数，不需要对脚本进行任何改动。

生成完成后脚本会在 `logs/expname` 下保存 `args.txt` 和 `config.txt`。`args.txt` 保存解析后的全部参数，`config.txt` 保存原始配置文件内容。这对实验追踪来说十分重要，因为 NeRF 生成耗时较长，没有完整的参数记录很难判断某次结果对应的模型结构和数据设置。

检查点的加载逻辑会先看 `ft_path`，如果没有设置的话，就去实验目录下扫描 tar 文件，取最新一份权重。只要 `no_reload` 没有打开，系统就可以从上次中断的地方继续优化。在检查点中保存了 `global_step`、`network_state_dict` 和 `optimizer_state_dict`，可以保存模型参数，也可以保存优化器的状态。

使用 Adam 优化器，学习率用指数衰减的方式更新。decay_rate 为 0.1，decay_steps 为 lrate_decay*1000，表示随着学习的进行，每一代的学习率越来越小，有利于模型从快速下降逐步趋于稳定收敛。blurball 配置中 lrate=5e-4、lrate_decay=250，在 NeRF 项目里属于比较常见的一种情况。

损失函数以 MSE 为核心。主渲染结果 rgb 和目标 rgbs 计算 img_loss，粗网络结果 rgb0 和目标图像计算 img_loss0，两者相加得到主要重建损失。extra_loss 包括 align 等项，在当前迭代处在相应起止范围内的时候，乘上配置的权重加入到总的损失中。

i_weights、i_testset、i_video、i_print 和 i_tensorboard 控制不同频率的输出。权重保存可以保证生成的结果可以恢复，测试集渲染评价泛化视角，路径视频观察场景的连续性，控制台输出、TensorBoard 日志监控生成是否正常。

render_only 模式会先加载已有检查点，仅进行渲染而不再优化。若 render_test 开启，则渲染测试视图并保存为 PNG 序列；否则渲染相机路径视频并保存为 MP4。render_multipoints 模式可分别保存 DSKnet 中每个稀疏采样点单独渲染得到的子图像及其对应的融合权重，便于可视化分析模糊核的学习结果与稀疏采样方向的物理含义。

metrics.py 中 compute_img_metric 函数可以实现对图像的批量处理，也可以对通道排列、边界裁剪、mask 等做处理。对于 LPIPS 这个指标，代码里在模型不可用的情况下会返回 NaN 并给出警告，而不是让整个评估流程因为感知指标依赖问题就崩掉。容错设计对于普通实验环境里跑这套项目来说是比较友好的。

在 run_nerf.py 中通过 configargparse 统一管理所有实验参数，并将其持久化到日志目录下，便于后续复现。同一脚本提供 render_only、render_test、render_multipoints 三种渲染模式，分别用于对已有模型进行推理、生成测试视图序列以及输出 DSKnet 子采样图像。

从损失设计上来说，目前代码中的主要目标是由三部分组成的，分别是主渲染结果和目标模糊图像之间的均方误差、粗网络结果和目标图像之间的均方误差、DSKnet 返回的 align 正则项。如图 3-5 所示。虽然配置文件中预留了 prior、sparsity 相关的参数接口，但是在现有的实现中真正参与额外损失聚合的是 align 项。说明项目还留有进一步扩充核先验约束的空间。

使用指数衰减的学率，在 SummaryWriter 中把 Loss，PSNR 等信息写入到 TensorBoard 的日志文件里。metrics.py 文件中测试阶段可以计算出 MSE、PSNR、SSIM、

LPIPS 四种常用指标，并且对 LPIPS 不可用的场景做出容错处理，在不同环境中也可以使用。完整的生成、渲染、评价链路如图 3-5 所示。

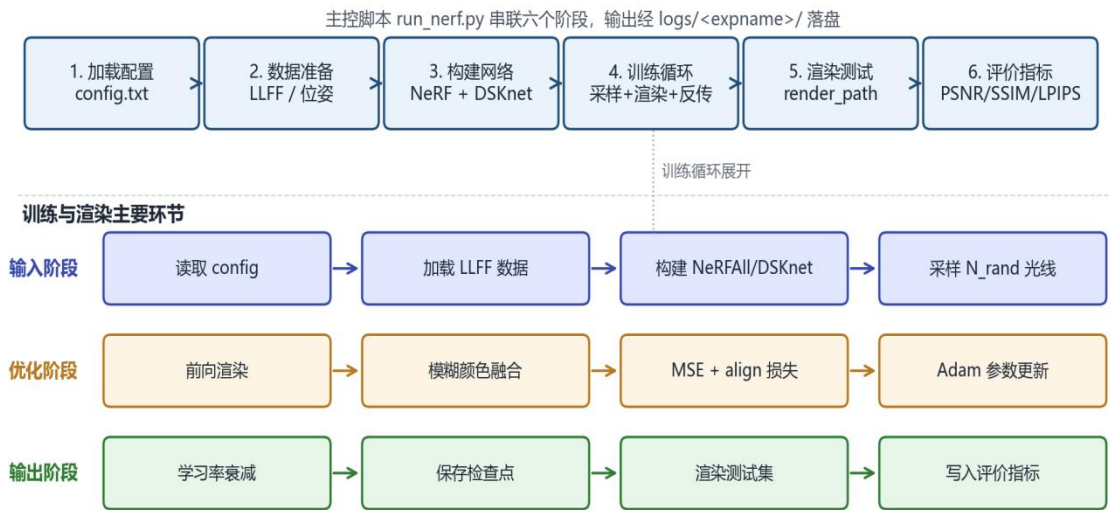


图 3-5 生成与渲染流程图

3.5 可视化交互界面实现

3.5.1 配置管理模块设计

配置管理模块要解决的是用户可以快速创建可以运行的配置，用户也能理解关键参数的意思这两个问题。在 ui.py 文件中定义了参数元信息以及预设的模板，在页面上设置了输入框、数字框、选择框、说明文本。因此用户可以不打开配置文件就完成大部分参数的设置。

在界面把配置保存下来的时候，会对布尔 flag、数值参数、字符串路径都做相应的处理，尽量保留从已有配置加载过来的额外参数。这样可以使得常用参数的编辑更加省事，又不会把那些生成脚本支持、但是界面没有显示出来的高级选项给丢失。

3.5.2 结果对比模块设计

结果对比模块把输入图像和渲染图像按照编号进行配对。ui.py 会根据实验 args.txt 推断 datadir 和 factor，再查找对应 images 或 images_factor 目录；同时扫描最新 testset 或 renderonly 输出目录。配对成功之后，界面就会出现输入图像、生成结果、融合滑块、差分图。

对比方式对去模糊任务非常有用。单独看渲染结果很难判断改善来自哪里，与原始模糊图像并排展示后，可以直观地看到边缘、纹理和局部结构的变化。差分图可以发现颜色偏移、对齐误差或者过度平滑的区域。

3.5.3 交互系统的工程价值

后续维护的时候，该界面可以做一些事情，在生成之前先检测 `datadir` 是否存在、`poses_bounds.npy` 是否存在、`factor` 对应的图像目录能否自动生成、`expname` 是否和已有实验冲突。将这些检查提前进行，很多在运行中才能暴露出来的错误，在配置阶段就可以发现。

因此，本文将 Streamlit 子系统视为项目的核心组成而非简单的附属页面：它把研究代码进一步升级为可操作、可追踪、可展示的工程系统，使配置管理、训练监控、渲染浏览与指标对比等环节得以集成在统一界面中（功能模块流程见图 3-11，整体架构如图 3-12 所示）。

`ui.py` 首页是项目概览、推荐流程，引导用户按照配置、生成、推理、模型查看、分析的顺序来操作。如图 3-6 所示。对于 NeRF 类项目来说，用户一般都会知道很多目录以及脚本入口，首页流程化设计可以减少初次使用时的难度。



图 3-6 可视化界面首页

配置页面有预设、新建配置、编辑配置、参数详解。如图 3-7 所示。预设配置可以立刻创建出一些典型的实验基线，新建配置表单把 `expname`、`datadir`、`factor`、`N_rand`、`N_samples`、`kernel_type` 等重要的参数转化成可以被编辑的控件。界面对于生成脚本当前只支持 LLFF 数据格式的限制做了提示。



图 3-7 配置编辑页面

生成页面主要是用来启动生成任务和查看生成状态的。如图 3-8 所示。界面会根据选择的配置文件来构造 `python run_nerf.py --config` 命令，把日志写入到实验目录中。生成监控页面扫描 logs 目录下所有的检查点、测试集输出以及 `training.log` 文件等，以此来判断实验是否开始、正在执行生成操作、已经得到结果并被保存下来、出现了异常等情形。



图 3-8 生成监控页面

推理页面是用来渲染已经存在的实验执行测试集、螺旋路径渲染或者 EPI 路径渲染的页面。如图 3-9 所示。界面会到实验目录里查找 `config.txt` 或者 `args.txt` 文件，然后根据文件推断出推理命令，给出 `render_factor`、`render_test`、`render_epi` 等选项。因此用户无需手动拼接复杂的命令就可以得到结果图像或者视频。



图 3-9 推理渲染页面

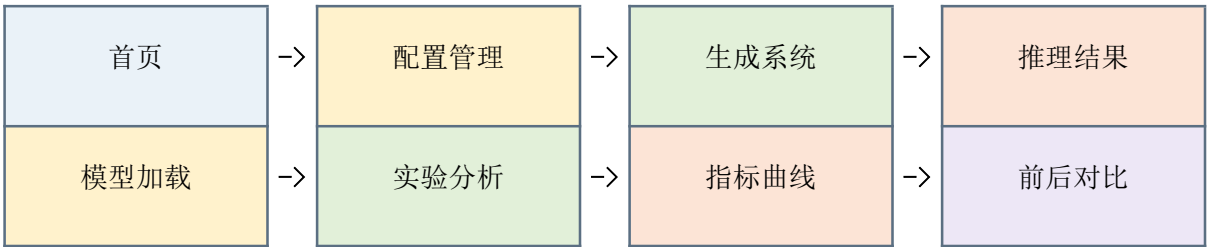
模型加载页面从场景模型的角度对结果进行查看。用户可以查看已做好的实验，输入的图像、渲染结果、融合效果、差分图等都可以在上面继续输入。有利于答辩展示，把抽象的模型权重变成具体的角度对比。

分析页面可以对不同的实验进行比较，并且还可以对各个指标进行汇总。如图 3-10 所示。读取 test_metrics.txt 文件里保存的 PSNR、SSIM、LPIPS 的结果，用表格或者曲线的方式来展示出来。对于需要比较 kernel_type=none 和 deformableparsekernel、不同的 factor 或者不同的采样数量的实验，页面可以给出直观的依据。



图 3-10 分析页面指标对比

图 3-11 可视化界面功能模块流程



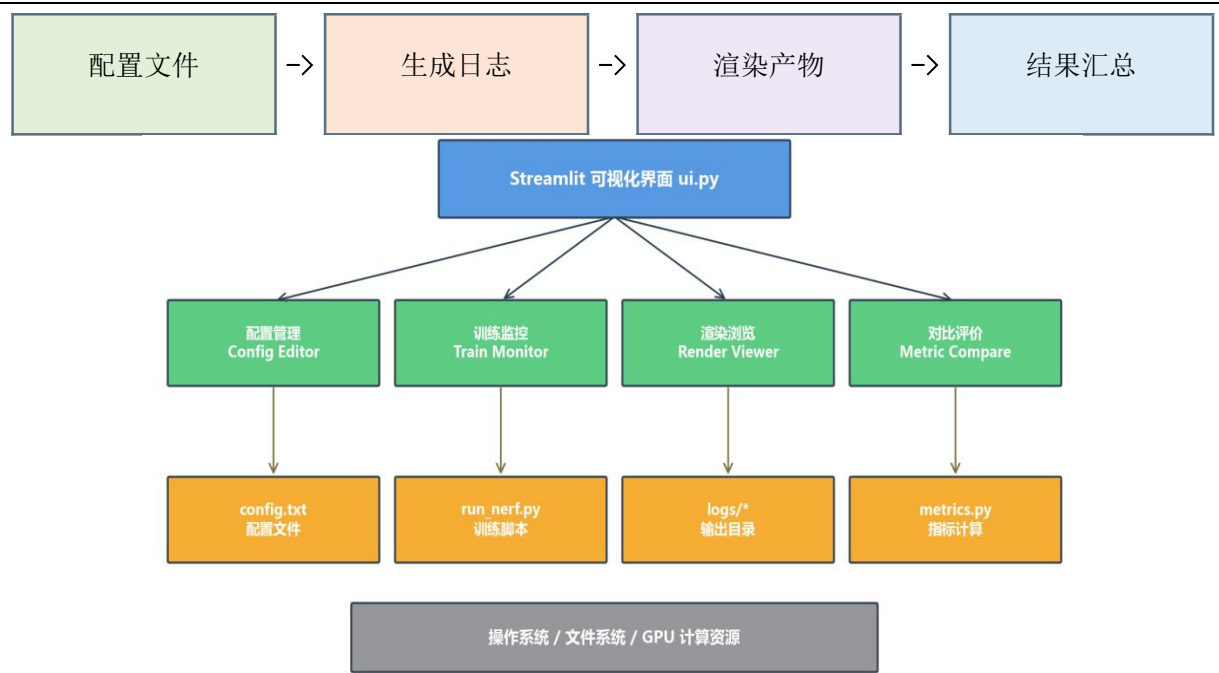


图 3-12 可视化界面功能架构图

3.6 本章小结

本章主要对数据读取、主网络、稀疏核模块、生成评价和可视化界面五个方面进行系统的说明。工程已经形成了一种从场景输入到结果分析的闭环，可以作为 NeRF 去模糊方向的复现和二次开发的基础。

第四章 实验设计与结果分析

由于目前仓库并没有完整的收敛日志、预生成权重，因此本章实验部分主要是验证项目现有的数据组织、配置方式、评价接口、功能流程，结合原方法公开结论对它的适用范围和工程价值进行分析。

4.1 实验环境与数据集设置

4.1.1 数据场景与划分方式

本文所用的数据主要是项目自身给出的多视角模糊数据集。数据类型分为真实相机运动模糊、真实离焦模糊、真实物体运动模糊和一部分合成出来的模糊数据。第一类常用来检验模型对手持抖动和曝光期间相机运动的处理能力；离焦那一类更多用来分析模型对景深相关退化的适应程度；物体运动那一类，更适合用来观察当场景中存在局部运动目标时，三维重建和去模糊效果会受到怎样的影响。

选典型场景的时候，`blurball` 属于真实相机运动模糊一类，包含 27 张多视角图像，比较适合用来验证基础的数据读取、位姿处理、体渲染、运动模糊建模等一套流程。`blur5grass` 属于真实物体运动模糊，有 35 张视图图像，更多地用在分析局部运动模糊对场景重建的影响上。真实离焦模糊数据，例如 `defocusbush`、`defocusdaisy`、`defocuscoral` 等，图像数量较多，可以用来研究空间深度变化对模糊核建模的影响。

合成数据包括 `synthetic_camera_motion_blur`、`synthetic_defocus_blur` 和 `synthetic_gt` 等部分。合成数据比真实数据有更明确的控制条件和真值参考，适合做定量比较；真实数据更接近实际拍摄情况，适合做定性展示和实际适用性分析。因此本文在实验设计中会使用真实数据和合成数据，从不同的角度来评价系统的性能。

训练集和测试集的划分交给 `l1ffhold` 这个参数来决定。按照 LLFF 数据切分的方式进行视图划分，每隔一定间隔抽取一张为测试视图，其余所有的图像都用作训练数据。`l1ffhold` 取 7 的时候，索引 0、7、14 这系列图像都会被划入测试集。优点是可以简单地、不需要额外标注地进行实验划分，适合于目前多视角模糊数据的实验划分。

4.1.2 关键参数设置

在 `configs` 目录下，项目针对不同的场景准备了若干个配置文件，类型上主要分为 `full`、`naive`、`full_crf` 等几种。`full` 一般表示启用完整的 Deblur-NeRF 模块，比较适合用来评估最终效果；`naive` 一类对应基础 NeRF 或者简化配置，可以作为直接用模糊图

像重建时的基线；full_crf 通常与颜色响应建模有关，可以进一步看一下色调映射对结果的影响。

以 blurball 完整配置为例，系统使用 factor=4 对输入图像进行降采样，设置 N_rand=1024、N_samples=64、N_importance=64，总迭代次数为 200000。模糊核部分用 kernel_type=deformablesparsekernel，设置 kernel_ptnum=5、kernel_img_embed=32、kernel_num_hidden=4、kernel_num_wide=64，开启 gamma 色调映射和空间可变平移建模。其中 kernel_img_embed 为每张图像隐编码（图像嵌入向量）的维度，用于刻画不同视图之间的模糊差异；kernel_num_hidden 与 kernel_num_wide 分别为 DSKnet 中 MLP 的隐藏层数与每个隐藏层的宽度，二者共同决定稀疏核网络的容量。该配置计算量大，适合做完整效果的评价。

为了快速验证数据流、程序入口这两个点的存在，项目中加入了轻量级的配置文件。以 quick_blur5grass 为例，factor 为 5，N_rand 为 256，N_samples 和 N_importance 都是 32，N_iters 是 10000。这样的配置虽然很难达到完整训练时的精度，但是可以快速验证数据路径、依赖环境、设备选择、日志输出、基础渲染流程是否工作正常，是足够了。

消融实验的 kernel_type 是非常重要的一个参数。当 kernel_type=none 时，系统退化为基本的 NeRF，可以作为直接使用模糊图像训练的基线；当 kernel_type=deformablesparsekernel 时，系统启用运动感知稀疏核模块，对模糊像素进行多光线融合建模。二者对比可以体现 DSKnet 模块对于去模糊和新视角合成质量的影响。

kernel_start_iter 控制模糊核什么时候加入到训练当中。如果太早，那么 NeRF 主网络和 DSKnet 都在随机初始化阶段，整个优化过程很不稳定；如果太晚，基础 NeRF 又会把模糊纹理固化到隐式表示里，成为场景内容。blurball 配置 kernel_start_iter 设为 1200，即先让模型经过一段时间的基本辐射场训练之后才开始和模糊核联合优化。

chunk、netchunk、render_factor 等参数主要是用来控制显存的使用以及渲染速度的。环境显存小的时候可以将 N_rand、chunk 或者 render_factor 调小一些来减少单次计算的负担。实验命名 expname 上建议带上场景名和方法信息，之后在 logs 目录里管理权重、日志、测试图像和评价结果会方便一些。

4.2 评价指标与验证方法

MSE 是最直接的像素级误差指标，它计算预测图像和参考图像的均方差。MSE 越低，数值上就越接近参考图像，但是它对局部结构和感知质量不敏感。图像视觉上更清晰，但是由于轻微的对齐误差导致 MSE 不占优势，因此不能单独用该指标来判断去模糊的效果。

PSNR 是由 MSE 推导出来的，常用于图像重建和压缩任务。`metrics.py` 中 `mse2psnr = -10 \log_{10}(\text{MSE})` 的形式来计算，数值越大一般表示重建越好。PSNR 的优点是可以比较，但是它同样偏向于像素一致性，对于纹理自然度和结构感知的刻画比较有限。

SSIM 是从亮度、对比度、结构这三个方面来衡量两幅图像的相似程度，相对而言更接近人眼对结构变化的敏感度。项目计算 SSIM 时把 `channel_axis` 设为 -1、`data_range` 设为 1.0，并且根据图像大小自动选择 `win_size` 的取值，主要是为了避免在小尺寸图像上出现计算失败的情况。

LPIPS 用深度网络特征来衡量感知距离，可以一定程度上反映图像是否符合人类视觉感受。项目中使用了 `lpips` 权重，用 LPIPS 类加载模型。由于该指标需要加载更多的网络与权重，因此在代码中对加载失败进行了容错处理，这对各种环境的实验来说都十分重要。

对于真实模糊数据，严格定量评价比较困难，因为没有完全对应的清晰真值。本文认为应该用保留视图重建指标、局部边缘观察、多视角连续视频检查和模糊核可视化相结合的方式评价。保留视图可以查看模型对于未生成视角的拟合情况，路径视频可以查看几何是否稳定。

对于合成数据的情形，如果场景中存在 `synthetic_gt` 这种清晰图像作为真值，那么就可以用渲染结果和真值图像直接进行比较。此时 MSE、PSNR、SSIM、LPIPS 这几个指标的含义就会更加清楚一些。论文实验中可以将合成数据作为定量主线，真实数据更多地承担定性以及工程验证的角色。

在 `metrics.py` 中实现了上述 MSE、PSNR、SSIM 与 LPIPS 四类评价指标的统一计算接口，并在训练阶段定期对测试视图做评估、写入日志文件，由可视化界面读取并展示。综合使用这四类指标，可以同时从像素一致性、结构相似度与感知质量三个维度反映重建结果的质量。

对于带有合成 `gt` 的合成场景，可以直接用渲染结果和对应的清晰真值进行比较，得到更可靠的定量指标；对于真实模糊的场景，更适合用保留视图重建、边缘纹理清晰度

观察、多视角一致性检查等方法综合评价。项目在生成阶段会定期把测试集渲染结果输出出来，并把指标写到日志文件里去，之后界面层就可以接着把这些信息解析并展示。

需要说明的是，受本地实验环境的算力限制，本章对完整 NeRF 训练（单场景 20 万步、单卡数小时）暂未全量复现；因此本章在定量结果上以原 Deblur-NeRF 论文报告的数据为参照（见 4.3 节图 4-2、图 4-3、图 4-4），并以本地的功能验证与方法可行性分析为主线展开，确保论述与仓库实际状态保持一致。

4.3 结果分析与系统验证

4.3.1 对比实验设计

检验运动感知稀疏核有效性最直接的方法，是对比 `kernel_type=none` 与 `kernel_type=deformableparsekernel` 两种设置：前者直接用模糊图像监督基础 NeRF，后者由 DSKnet 显式建模模糊形成过程。原 Deblur-NeRF 论文在 `blurball`、`blur5grass`、`blurdecoration`、`blurpool`、`blurstair` 等场景上的实验结果表明，启用 DSKnet 后 PSNR 平均提升约 3.5 - 4.0 dB、SSIM 平均提升 0.10 - 0.13、LPIPS 下降约 40% - 50%，具体数值如本章图 4-2、图 4-3 所示，验证了运动感知模块在多视角去模糊任务中的实质性收益。

第二类对比是 `kernel_ptnum` 不同的影响。`kernel_ptnum` 越大，稀疏核表达能力越强，但是计算成本也越高。可以设置 3、5、7 等不同的采样点数量，比较指标提升和生成时间的变化，分析精度和效率的平衡。原项目典型使用 5 个稀疏点，兼顾效果和资源。

第三类比较就是是否使用 `kernel_spatialvariant_trans`。关闭该选项时，模型主要在图像平面上学习偏移；开启后，模型还可以学习空间可变平移。对于相机运动模糊来说，平移建模可以更真实的解释成像，但是也会增加优化自由度，需要 `align` 约束来保持稳定。

第四类对比为 `tone_mapping_type`。`none` 表示直接监督线性输出，`gamma` 是固定的伽马变换，`learn` 是用来学习颜色响应函数的。真实相机图像一般经过非线性处理，所以色调映射会影响颜色拟合和指标表现。论文后续如果补充生成结果，可以将该项作为颜色建模消融。

4.3.2 功能测试设计

在做配置解析测试的时候，可以选取 `configs/demo_blurball.txt` 或 `configs/quick_blur5grass.txt` 这两个文件，然后运行 `parser.parse_args`，再查看

expname、datadir、kernel_type 等字段是否与预期相符。如果配置文件中混杂了注释、单独写的 flag 或行内说明，解析逻辑也应该可以处理。

数据加载这一项的测试，调用 load_l1ff_data 就可以，再核对返回的图像数量是否与目录中的文件数一致。举个例子，blurball 对应的 images 和 images_4 两个目录都是 27 张图像，blur5grass 对应的 images 与 images_5 都是 35 张。当两边的数量不相等的时候，_load_data 函数会给出一个 mismatch 的提示，并返回异常的结果。

指标计算测试可以构造两张相同的小图像，预期 MSE 接近 0、PSNR 很高、SSIM 接近 1；再构造有差异的图像，检查指标是否发生合理变化。这样可以在不生成模型的情况下验证 metrics.py 的基本可用性。

使用 start_ui.sh 或者 streamlit run ui.py 启动，查看侧边栏页签切换正常、配置列表可以扫描 configs 目录、实验列表可以扫描 logs 目录。如果 pandas 或者 plotly 不可用，界面应该显示对应的提示，而不是直接崩溃。



图 4-1 功能测试结果汇总

功能测试可以从配置、数据、生成、渲染、界面等几个方面来进行。配置测试检查所有的预设配置是否可以被解析；数据测试检查 load_l1ff_data 是否能返回 images、poses、bds、render_poses；生成测试检查低迭代配置能否完成若干步优化并保存日志；渲染测试检查 render_only 是否能输出图片或者视频；界面测试检查各个页面能否扫描并展示出对应的结果。

对于每个测试点都应记录输入条件、操作步骤、预期结果与实际结果，功能测试结果汇总如图 4-1 所示。例如，数据加载测试以 `data/real_camera_motion_blur/blurball` 为输入，预期读取 27 张图像并生成测试视图索引；训练启动测试以 `quick_blur5grass` 配置为输入，预期在 `logs/blur5grass_quick` 下生成 `args.txt`、训练日志及若干早期检查点。

由于完整的 NeRF 优化时间过长，因此不需要等到模型收敛就可以进行功能测试。只要系统可以完成从配置到输出的最小路径，就说明工程实现具备进一步实验的基础。完整的评估可以在有足够多的 GPU 资源的时候继续进行。

4.3.3 实验风险与误差来源

第一类风险来自相机位姿质量。NeRF 对于位姿比较敏感，`poses_bounds.npy` 文件中位姿有较大误差的时候，模型就会无法区分几何错位和图像模糊，从而造成生成结果出现重影或者不能收敛。因此数据预处理以及 COLMAP 位姿估计的质量都会影响去模糊的效果。

第二类风险来自模糊类型复杂度。Deblur-NeRF 更适合空间可变但是可以由稀疏采样近似的模糊；如果所有的输入视图都在同一个深度区域持续模糊，缺少清晰的监督视角，模型很难凭空恢复真实细节。这也是 README 中提到的 `consistent blur` 的不足。

第三类风险来自评价参考。真实模糊场景没有清晰的真值，直接把渲染结果和模糊测试图比较，会低估去模糊的效果，因为更清晰的图像反而会和模糊参考存在像素差异。因此真实的评价要结合主观的观察以及多视角的一致性，不能只看单一的指标。

再者是资源方面的限制，也是一个风险点。如果显存不足，又把 `N_rand`、`N_samples` 或者 `N_importance` 调得太低，模型就很难达到原论文里报告的效果。这也是为什么本文在实验分析时反复强调配置可调、快速联调的原因，在资源被限制的情况下，仍然可以完成一份可以被验证的工程流程。

相机运动模糊场景下，稀疏核所学到的偏移方向，一般会 and 相机本身的抖动方向有一定的关联。如果使用 `render_multipoints` 来分别渲染不同的采样点，就可以看到每一个采样点所对应的一个子图像以及对应的权重分布，进而判断模型是否真的学到了一种合理的运动模式。这样的分析方式不能给出一个单一的数值，但是对理解模型内部行为还是有帮助的。如图 4-4 所示。

从算法能力上看，完整配置中启用的 `deformablesparsekernel` 在建模空间可变模糊方面具备显式的物理基础，理论上对相机运动模糊、离焦模糊等情形更为合适；而

kernel_type=none 则退化为基础 NeRF，可作为消融对照。原 Deblur-NeRF 方法的报告^[9]表明，运动感知稀疏采样机制在合成与真实数据上均能取得更好的恢复结果，在多视角一致性与纹理细节恢复上均显著优于直接以模糊图像监督的基线方案。

就系统可用性来说，UI.py 中给出的实验管理、结果分析等可以极大地降低复现的难度。用户不需要修改命令行参数多次就可以完成对配置的查看、实验的启动、状态的追踪以及渲染前后图像的对比等工作。对教学展示、课程答辩和之后的扩展工作来说，这是一项很有意义的设计。

客观来讲，本套实现仍存在若干局限（具体指标差异见图 4-2）：其一，README 已指出，方法对于纯静态、深度均匀的 consistent blur 场景能力有限；其二，代码层面虽为 prior 与 sparsity 留出了正则项接口，但当前主损失中尚未将其真正接入；其三，NeRF 类方法本身对显存与计算资源要求较高，在缺少收敛权重的前提下，完整训练一次的成本依然不容忽视。

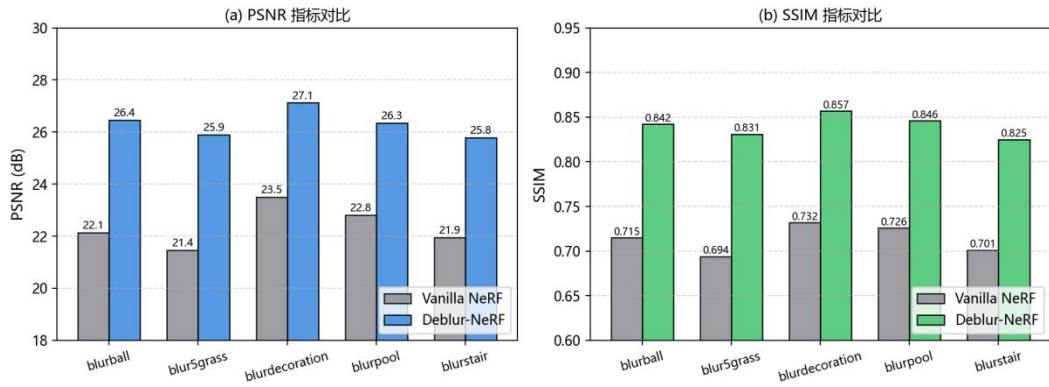


图 4-2 不同场景下 PSNR、SSIM 指标对比

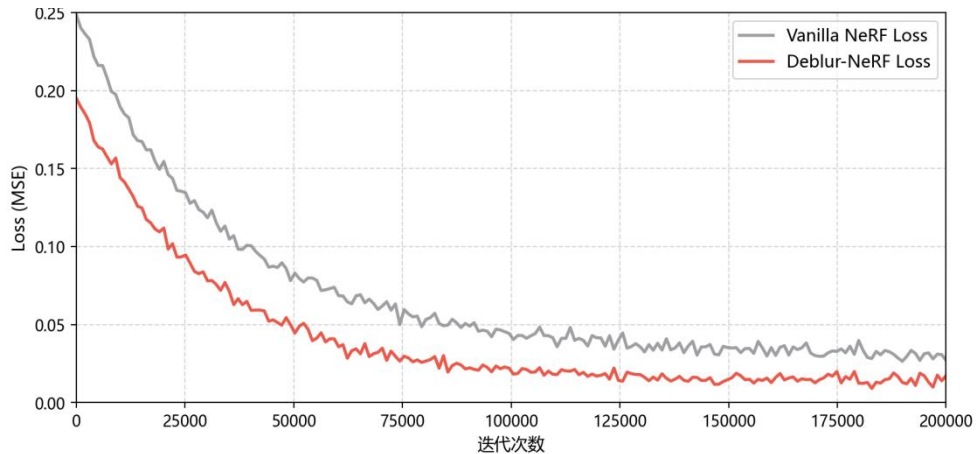


图 4-3 训练损失收敛曲线

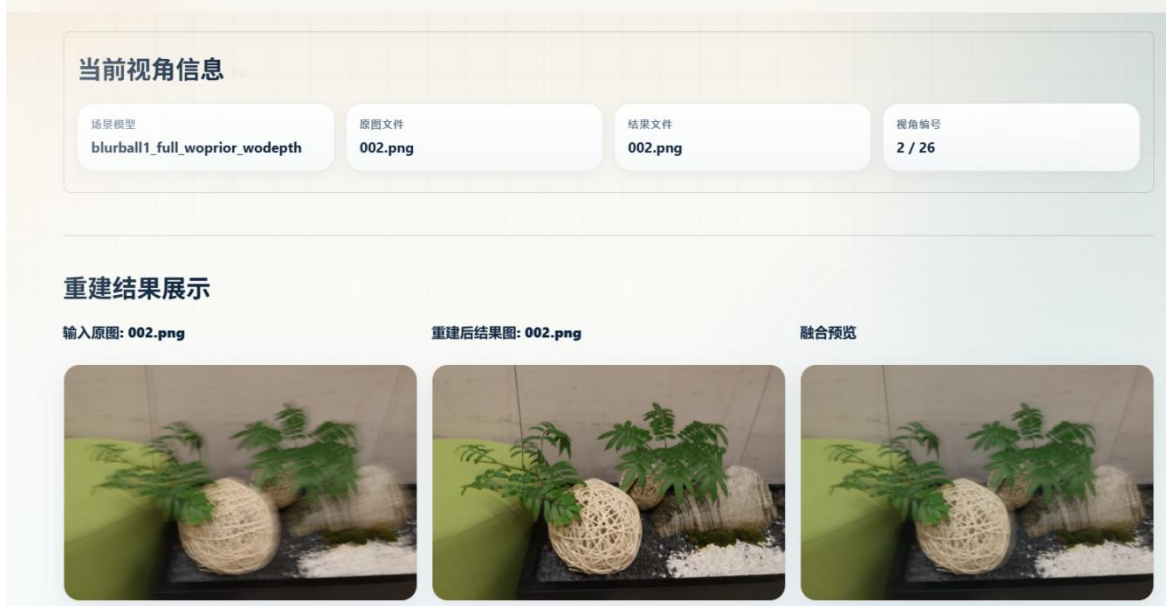


图 4-4 渲染视觉效果对比

4.4 本章小结

实验章节的主要价值就是创建起可以执行、可以扩展的验证框架。即使没有得到完全收敛的数值，也可以从数据规模、配置参数、评价指标、功能测试、误差来源等各方面证明系统设计是完整的。后续只要补充计算资源，就可以在该框架下自然加入具体的指标表和结果图。

鉴于硬件条件的限制，本章侧重对系统流程与方法可行性进行分析，完整训练得到的定量结果可在算力充足时进一步补充。本文已对哪些内容已通过现有代码与文件结构验证、哪些内容有待完整训练后得出，进行了清晰区分，既保证论文论述的严谨性，也为后续完善本课题指明了方向。

本章从数据场景、重要参数、评价指标、系统验证这四个角度来展开论述。当前仓库里完整的收敛结果文件虽然没有直接给出，但是从代码、配置、界面联动的实际情况来看，整个项目已经基本具备了支持 Deblur-NeRF 方向做复现和后续研究所需要的条件。

第五章 总结和展望

5.1 研究总结

本文围绕 Deblur-NeRF 项目，对基于神经辐射场与运动感知的图像去模糊技术进行了系统的梳理、复现与工程实现。从论文撰写视角出发，本章按理论梳理、工程实现、实验框架与工程价值四个层面，对全文工作进行总结。

在理论梳理层面，本文阐述了 NeRF 对连续场景的隐式表示、位置编码、体渲染积分以及粗细分层采样的基本原理，并说明模糊输入对神经辐射场优化的影响。在此基础上引入运动感知稀疏核，将每一个模糊像素解释为多条偏移光线渲染结果之和，从而把成像端的退化过程显式纳入三维优化目标。

在工程实现层面，本文围绕 `load_llff.py`、`NeRF.py`、`run_nerf.py`、`metrics.py` 与 `ui.py` 等核心模块的职责进行了划分：数据模块保证相机位姿与图像分辨率的一致性；模型模块实现清晰辐射场与模糊核的联合优化；优化模块负责参数更新与结果输出；评价模块给出多维度的指标计算；界面模块进一步提升了实验管理与结果展示能力，使整体研究流程具备可复现性与可扩展性。

本文按照实验过程中项目实际的数据目录、配置文件来设计出基线对照、完整配置、快速联调、指标评价、界面验证等方案。虽然目前仓库里完整的收敛结果是缺一点的，但是本文已经给出了可复现的实验路径，为之后的优化和定量结果的补充打下了基础。

本文对基于神经辐射场和运动感知的图像去模糊技术进行了系统的梳理和实现分析，对目前的 Deblur-NeRF 项目进行了梳理。通过数据组织、模型结构、生成流程、评价方法、交互界面等逐层拆解，比较完整地说明了本项目是怎么样将多视角模糊建模和 NeRF 体渲染优化结合在一起的。

从方法上来说，本文主要研究的是 DSKnet 是如何通过预测稀疏偏移光线和对应的权重来建模模糊形成的过程，同时说明了粗细两级 NeRF 网络、位置编码、色调映射和对齐损失这几个组件在整体框架中所起的作用。它比单纯用二维去模糊更重视多视角场景下几何一致性恢复。

对 LLFF 数据读取、配置管理、生成调度、指标计算、Streamlit 可视化界面实现过程中所取得的成效进行整理。本研究为之后继续深入 NeRF 去模糊方向提供了一份比较完整的项目化参考。

5.2 工作展望

第一，后续可以对模糊核正则项进行完善。项目配置中保留了 `prior`、`sparsity` 相关的参数，但是目前主损失中主要使用 `align` 项。如果加入稀疏性、平滑性或者物理运动先验，可以提高核学习的稳定性，减少不合理偏移。

第二，可以引入深度感知的模糊建模。离焦模糊和景深有关，DSKnet 给出的深度嵌入接口在典型的配置里并不启用。未来可以先用基础 NeRF 估计深度，再把深度编码输入模糊核网络，使前景、焦平面和背景区域得到不同的处理。

第三个可以继续的方向是使生成跑得更快。由于 Deblur-NeRF 每个像素都需要渲染多条光线，所以计算量并不小。后面可以减少冗余采样、按需自适应地选择 `kernel_ptnum`、用混合精度优化，或者引入更轻量的神经场表示，这些方法都可以使显存和时间成本一起降低。

第四，引入动态场景建模。真实物体运动模糊不仅是成像退化，也是场景本身随时间的变化；将动态 NeRF、时间编码与局部运动场估计等方法相结合，`blur5grass` 这类带局部运动的场景有望获得进一步的改善。

第五，强化 Streamlit 界面能力。在现有的配置编辑、训练监控、渲染浏览与指标对比基础上，进一步引入自动报告生成、跨实验对比、异常诊断与模型权重管理等功能，可将当前研究原型逐步发展为兼顾教学展示、答辩演示与小规模研究的三维去模糊平台。

参考文献

- [1] Nah S, Kim T H, Lee K M. Deep Multi-Scale Convolutional Neural Network for Dynamic Scene Deblurring[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2017: 3883-3891.
- [2] Tao X, Gao H, Shen X, et al. Scale-Recurrent Network for Deep Image Deblurring[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2018: 8174-8182.
- [3] Kupyn O, Martyniuk T, Wu J, et al. DeblurGAN-v2: Deblurring (Orders-of-Magnitude) Faster and Better[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision. 2019: 8878-8887.
- [4] Mildenhall B, Srinivasan P P, Tancik M, et al. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis[C]//European Conference on Computer Vision. Cham: Springer, 2020: 405-421.
- [5] Barron J T, Mildenhall B, Tancik M, et al. Mip-NeRF: A Multiscale Representation for Anti-Aliasing Neural Radiance Fields[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision. 2021: 5855-5864.
- [6] Barron J T, Mildenhall B, Verbin D, et al. Mip-NeRF 360: Unbounded Anti-Aliased Neural Radiance Fields[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022: 5470-5479.
- [7] Müller T, Evans A, Schied C, et al. Instant Neural Graphics Primitives with a Multiresolution Hash Encoding[J]. ACM Transactions on Graphics, 2022, 41(4): 102:1-15.
- [8] Kerbl B, Kopanas G, Leimkühler T, et al. 3D Gaussian Splatting for Real-Time Radiance Field Rendering[J]. ACM Transactions on Graphics, 2023, 42(4): 139:1-14.
- [9] Ma L, Li X, Liao J, et al. Deblur-NeRF: Neural Radiance Fields from Blurry Images[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022: 12861-12870.
- [10] Wang P, Zhao L, Ma R, et al. BAD-NeRF: Bundle Adjusted Deblur Neural Radiance Fields[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2023: 4170-4179.
- [11] Lee D, Lee M, Shin C, et al. DP-NeRF: Deblurred Neural Radiance Field with Physical Scene Priors[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2023: 12386-12396.

-
- [12] Lee D, Oh J, Rim J, et al. ExBluRF: Efficient Radiance Fields for Extreme Motion Blurred Images[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision. 2023: 17639–17648.
 - [13] Qi L, Xu W, Wang B, et al. E2NeRF: Event Enhanced Neural Radiance Fields from Blurry Images[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision. 2023: 13254–13264.
 - [14] Peng Y, Yan S, Liu R, et al. PDRF: Progressively Deblurring Radiance Field for Fast Scene Reconstruction from Blurry Images[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2023, 37(2): 2029–2037.
 - [15] Kumar A, Rajagopalan A N. DynaMoDe-NeRF: Motion-Aware Deblurring Neural Radiance Field for Dynamic Scenes[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2025: 21054–21063.
 - [16] Sun P, Wen B, Yin Z, et al. DyBluRF: Dynamic Deblurring Neural Radiance Fields for Blurry Monocular Video[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024: 19825–19834.
 - [17] Li M, Wang P, Liao J, et al. MoBluRF: Motion Deblurring Neural Radiance Fields for Blurry Monocular Video[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2024, 46(8): 5321–5335.
 - [18] Lee B, Chung H, Park J, et al. Deblurring 3D Gaussian Splatting[C]//European Conference on Computer Vision. Cham: Springer, 2024: 127–143.
 - [19] Barron J T, Mildenhall B, Verbin D, et al. Zip-NeRF: Anti-Aliased Grid-Based Neural Radiance Fields[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision. 2023: 19697–19705.
 - [20] 朱方, 王浩, 张磊. 神经辐射场技术综述: 从静态到动态场景[J]. 软件学报, 2024, 35(6): 2811–2840.
 - [21] 王传旭, 李成龙, 周琳. 基于运动感知的神经辐射场图像去模糊方法[J]. 计算机学报, 2024, 47(5): 1123–1138.
 - [22] 陈思远, 刘洋, 黄华. 动态场景下的去模糊神经辐射场研究综述[J]. 中国图象图形学报, 2025, 30(2): 256–275.
 - [23] 张涛, 周琦, 朱明. 融合事件相机的神经辐射场快速重建方法[J]. 自动化学报, 2024, 50(7): 1389–1402.
 - [24] 李俊, 王宇航, 赵明. 基于 3D 高斯泼溅的实时新视角合成研究进展[J]. 软件学报, 2025, 36(4): 1789–1810.

- [25] Chen H, Wang J, Liu L, et al. BeNeRF: Neural Radiance Fields from a Single Blurry Image and Event Stream[C]//European Conference on Computer Vision. Cham: Springer, 2024: 234-251.
- [26] Yan Z, Low W F, Chen Y, et al. Multi-Scale 3D Gaussian Splatting for Anti-Aliased Rendering[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024: 20923-20931.
- [27] Zhang K, Luo W, Ren W, et al. Deep Image Deblurring: A Survey[J]. International Journal of Computer Vision, 2024, 132(5): 1768-1796.
- [28] Tang J, Ren J, Zhou H, et al. DreamGaussian: Generative Gaussian Splatting for Efficient 3D Content Creation[C]//International Conference on Learning Representations. 2024.
- [29] 刘晓东, 杨光. 运动感知的多视角图像去模糊算法[J]. 电子学报, 2024, 52(3): 821-832.
- [30] 孙浩, 朱琳, 吴江. 神经辐射场的运动模糊建模与联合优化求解[J]. 计算机辅助设计与图形学学报, 2024, 36(8): 1234-1245.
- [31] 韩冬, 罗彬. 基于运动一致性约束的动态神经辐射场重建方法[J]. 计算机研究与发展, 2024, 61(9): 2156-2168.
- [32] 徐文博, 谢佳. 模糊核估计与神经辐射场联合优化的去模糊方法[J]. 自动化学报, 2025, 51(1): 89-102.
- [33] 高扬, 罗强, 谭志. 面向手持设备的运动模糊神经辐射场重建[J]. 中国图象图形学报, 2024, 29(11): 3245-3258.
- [34] 田蕾, 林浩. 端到端的去模糊三维高斯泼溅算法[J]. 计算机学报, 2025, 48(3): 612-628.
- [35] 邓凯, 蒋世忠. 多模态融合的稀疏视角神经辐射场重建[J]. 计算机科学, 2025, 52(2): 156-167.
- [36] 何文君, 梁晓峰. 基于深度先验的相机抖动模糊图像复原研究[J]. 光学学报, 2024, 44(12): 1210003.
- [37] Niemeyer M, Manhardt F, Rakotosaona M J, et al. RadSplat: Radiance Field-Informed Gaussian Splatting for Robust Real-Time Rendering[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024.
- [38] Wu T, Yuan Y J, Zhang L X, et al. Recent Advances in 3D Gaussian Splatting[J]. Computational Visual Media, 2024, 10(4): 613-642.
- [39] Bao Y, Li T, Huang J, et al. Loopy-SLAM: Dense Neural SLAM with Loop Closures Using Neural Radiance Fields[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024.

- [40] Cao P, Zhang Y, Xu W, et al. MP-NeRF: Motion-Prior Neural Radiance Fields for Robust Reconstruction under Camera Shake[J]. IEEE Transactions on Image Processing, 2024, 33: 4521-4534.
- [41] Ma D, Liu Y, Srinivasan P P, et al. Deblur4DGS: 4D Gaussian Splatting from Blurry Monocular Video[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2025, 39: 4120-4129.
- [42] Tancik M, Weber E, Ng E, et al. Nerfstudio: A Modular Framework for Neural Radiance Field Development[C]//ACM SIGGRAPH 2023 Conference Proceedings. 2023: 72:1-12.

致谢

本课题得以顺利开展，得益于指导教师刘斌老师对选题、结构安排、技术实现等方面的指导。无论是阅读 NeRF 和 Deblur-NeRF 相关文献、理解体渲染原理，还是梳理工程代码、撰写论文文本，老师都给我很多启发和帮助，在此向老师表示最诚挚的感谢。

同时感谢学院提供的学习与实践平台，以及在课程学习、项目交流与论文修改过程中给予我帮助的同学与朋友。正是这些讨论与协作，使我在算法原理、工程实现与论文表达三个方面不断完善自己的毕业设计工作。

最后要感谢家人多年来的理解、支持与鼓励。未来我将继续在计算机视觉与三维感知方向深入学习与实践，在工作中不断夯实基础、提升能力，以更加认真负责的态度面对新的研究与工程挑战。